

软件实习

智慧图书自习室管理系统开发报告

课程名称：	软件实习
系统名称：	智慧图书自习室管理系统
学期：	2023-2024 第二学期
任课老师：	辛宇
所在学院：	信息科学与工程学院
专业班级：	21 级计算机科学与技术 1 班
队长：	赵瑞杰　216001944
队员：	田礼凯　216001927
队员：	郭力源　216001902
队员：	刘斌辉　216001916

日期：2024 年 6 月 19 日

目录

一、 项目概述 3
二、 项目规划 3
2.1 项目背景 3
2.2 需求分析 4
三、 项目设计 5
3.1 类设计 5
3.1.1 类关系 5
3.1.2 Entity 类 7
3.1.3 Mapper 类 7
3.1.4 Service 类 7
3.1.5 Controller 类 10
3.2 接口设计 12
3.2.1 用户接口设计 12
3.2.2 预约接口设计 15
3.2.3 座位接口设计 18
3.2.4 自习室接口设计 22
3.2.5 违规接口设计 24
3.2.6 图书接口设计 27
四、 软件展示 30
4.1 登录界面 30
4.1.1 用户登录 30
4.1.2 用户注册 32
4.2 用户界面 34
4.2.1 自习室预约 34
4.2.2 预约信息查询 36
4.2.3 签到签退 37
4.2.4 图书借阅 39
4.3 管理员界面 40
4.3.1 自习室管理 40
4.3.2 座位管理 43
4.3.3 预约管理 45
4.3.4 用户管理 48
4.3.5 违规管理 50
4.3.6 图书管理 55
五、 实验总结 57

一、项目概述

本次项目基于敏捷开发方法以及 DevOps 思想，借助华为云 CodeArts 开发平台实践完成了对智慧图书自习室管理系统的设计开发与集成部署，该项目旨在解决传统图书馆管理中存在的问题，如座位资源利用不充分、图书借阅流程麻烦和管理效率低下等问题，通过智能化系统提升图书馆自习空间资源的使用效率，减少学生等待时间，并实现图书自习室的高效管理。

在技术实现上，系统后端采用 Springboot 框架，遵循三层架构模式（Controller、Service、Mapper），并使用 MyBatisPlus 进行数据库操作。前端则采用 Vue.js 框架，提供了用户友好的界面和交互体验。系统实现了用户登录、注册、预约座位、查看座位信息、预约信息查询、借书、还书、书本信息管理、管理员信息管理、违规信息处理等功能。

本项目使用华为云 Codearts 平台完成了项目开发中代码托管、持续开发、代码检查、编译构建、部署与持续集成、持续测试等步骤，确保了代码的质量和系统的稳定性，并且通过流水线将代码检查、编译构建和部署流程与代码仓库关联自动化，提高了开发效率和产品质量。其中，使用华为云 Codearts 平台与 ECS 弹性云服务器部署的智慧图书自习室管理系统对公网开放的服务地址为 http://124.71.194.156/。

最终，智慧图书自习室管理系统交付了一个高效、稳定且用户友好的软件产品，满足了校园内数字化和智能化转型的需求，为学生提供了便捷、高效的学习环境，同时也为校园服务的数字化转型贡献了力量。

二、项目规划

2.1 项目背景

随着社会信息化的不断深入，校园内的各项服务正经历着数字化和智能化的全面转型。在这个不可避免的趋势下，智慧图书自习室管理系统崭露头角。传统的图书馆管理方式存在着一系列难题，例如座位资源难以有效利用、管理效率不高等问题层出不穷。为了迎接时代潮流，我们决定着手设计一套智能的座位预约和图书借阅系统，旨在提升图书馆资源的充分利用率，减少学生的等待时间，同时实现对图书馆的高效管理。

在当今大学生活中，图书馆被认为是学习和思考的重要场所，因此图书馆的合理利用对学生的学业产生着直接而重要的影响。为了更好地满足学生的学习需求，提高学习效率，我们力求通过设计智慧图书自习室管理系统，实现图书馆座位的智能预约、快速借阅书籍、统计分析等一系列功能。这样的系统将为学生提供更为便捷、高效的学习环境，帮助他们更好地规划学习时间，提升学术表现。这不仅有助于解决传统管理方式存在的问题，更为校园服务的数字化转型贡献一份力量。

2.2 需求分析

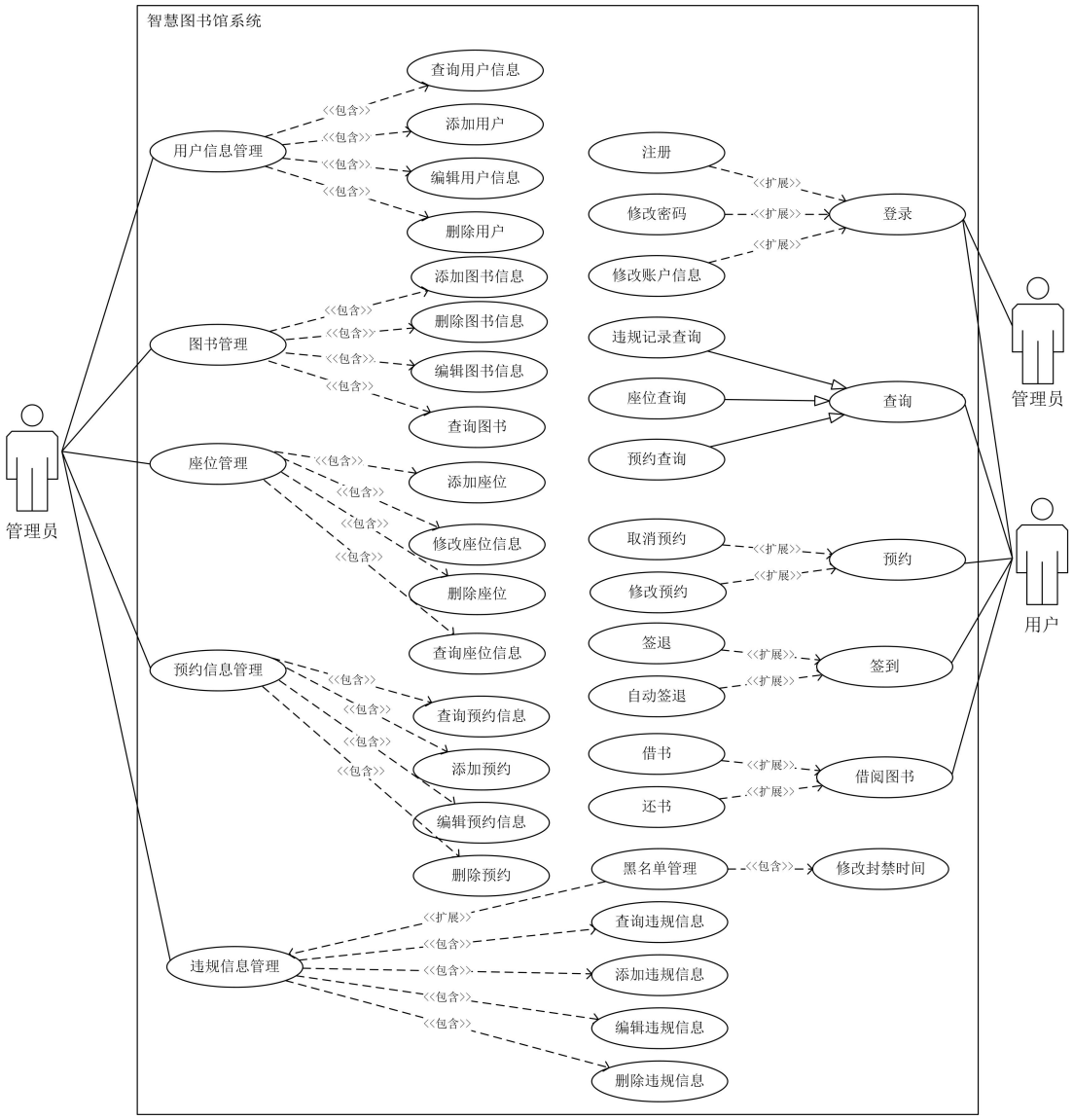


图 1 用例图

我们首先创建两个参与者，管理员和用户，他们共有的用例为“登录”，在登陆时进行鉴权，用户进入预约自习室界面，管理员进入管理界面。然后我们考虑用户所需要的基本功能为预约，签到，查询，借阅图书，所以我们设置了“预约”，“签到”，“查询”，“借阅图书”三个用例，并在此基础上延伸出，“修改预约”，“自动签退”等扩展用例。

考虑完用户后，我们考虑管理员，管理员肯定有对各种信息的管理能力，所以我们根据用户一整套预约流程所需要的信息，设计了管理员的用例：

“用户信息管理”：管理员可以对用户账号信息进行增删改查操作。

“图书管理”：管理员可用添加图书信息，查看该图书被那位用户借阅，该用户应该还书的时间，还可以删除图书，查询图书。

“座位管理”：管理员可以在自习室中添加座位，可以查看当前座位是否被占用，被那位用户预约，如果座位出现故障管理员可以将该座位从数据库中删除，避免被用户预约。

“预约信息管理”：管理员可以帮助用户进行预约座位的操作，还可以修改用户预约信息，如果该用户违规，管理员还可以手动删除该用户预约消息。

“违规信息管理”：管理员可以手动添加用户违规消息；管理员可以删除用户违规消息；管理员可以查看用户的违规时间、违规次数和封禁时间，如果用户申诉通过可以缩短用户封禁时间。

三、项目设计

根据需求分析报告以及设计报告，能够对于智慧自习图书室预约管理系统的实现有了初步筹划，在本次项目的实现过程中，后端采用了 Springboot 的框架，遵循三层基本架构模式（Controller、Service、Mapper），将职责分配明确，并根据基本实体类进行代码构筑。

3.1 类设计

SpringBoot 是一个基于 Spring 框架的开源 Java 基础框架，它简化了基于 Spring 的应用开发，提供了各种默认配置，在本次项目中配合 Spring MVC 使用来构建 Web 应用程序。

在 Spring Boot 应用中，传统的三层架构（或称为 MVC 模式）指的是如下三层。

(1) 表示层（Presentation Layer）：

这一层通常由 Spring MVC 的 Controller 类组成，负责处理 HTTP 请求和响应。Controller 层是用户与应用交互的接口，它将用户的输入转换为对业务逻辑层的调用，并最终返回响应结果。

(2) 业务逻辑层（Service Layer）：

业务逻辑层包含应用的核心业务逻辑。在 SpringBoot 应用中，这一层通常由 Service 类实现。Service 类可以调用数据访问层来获取或存储数据，也可以包含一些业务规则的实现。

(3) 数据访问层（Data Access Layer）：

数据访问层负责与数据库交互，执行 CRUD（创建、读取、更新、删除）操作。在 Spring Boot 中，这一层通常由 Repository 接口或 JPA（Java Persistence API）的 EntityManager 实现。MyBatis 也是一个常用的 ORM 框架，通过 Mapper 接口与数据库进行交互。在本次项目实现过程中采用 MyBatisPlus 实现 Mapper 接口与数据库进行交互。

3.1.1 类关系

除了传统的三层架构框架之外，还包含最基础的实体类，本次项目的设计实现 UML 类图如图 3 所示，从左至右，每一列分别代表实体类层（Entity）、数据访问层（Mapper）、业务逻辑层（Service）以及表示层（Controller）。

实体类之间的关系如下所示：

(1) 聚合关系：Room 包含 List<Seat>，一个自习室由多个座位组成。
(2) 关联关系：Reserve 关联到 User，一个预约由一个用户创建。
(3) 关联关系：Reserve 关联到 Room，一个预约发生在一个特定的自习室内。
(4) 关联关系：Reserve 关联到 Seat，一个预约涉及到一个特定的座位。
(5) 关联关系：SeatTime 关联到 Seat，座位时间与特定座位相关。
(6) 关联关系：SeatTime 关联到 Reserve，座位时间记录了预约的时间段。
(7) 关联关系：Log 关联到 User，Log 记录了用户的行为。

实体类与 Mapper 之间的关系如下所示：

(1) 依赖关系：UserMapper 接口依赖于 User，UserMapper 接口包含操作用户数据表的方法。
(2) 依赖关系：RoomMapper 接口依赖于 Room，RoomMapper 接口包含操作自习室数据表的方法。
(3) 依赖关系：ReserveMapper 接口依赖于 Reserve，ReserveMapper 接口包含操作预约数据表的方法。

(4) 依赖关系：SeatMapper 接口依赖于 Seat，SeatMapper 接口包含操作座位数据表的方法。
(5) 依赖关系：SeatTimeMapper 接口依赖于 SeatTime，SeatTimeMapper 接口包含操作座位时间数据表的方法。
(6) 依赖关系：LogMapper 接口依赖于 Log，LogMapper 接口包含操作日志数据表的方法。

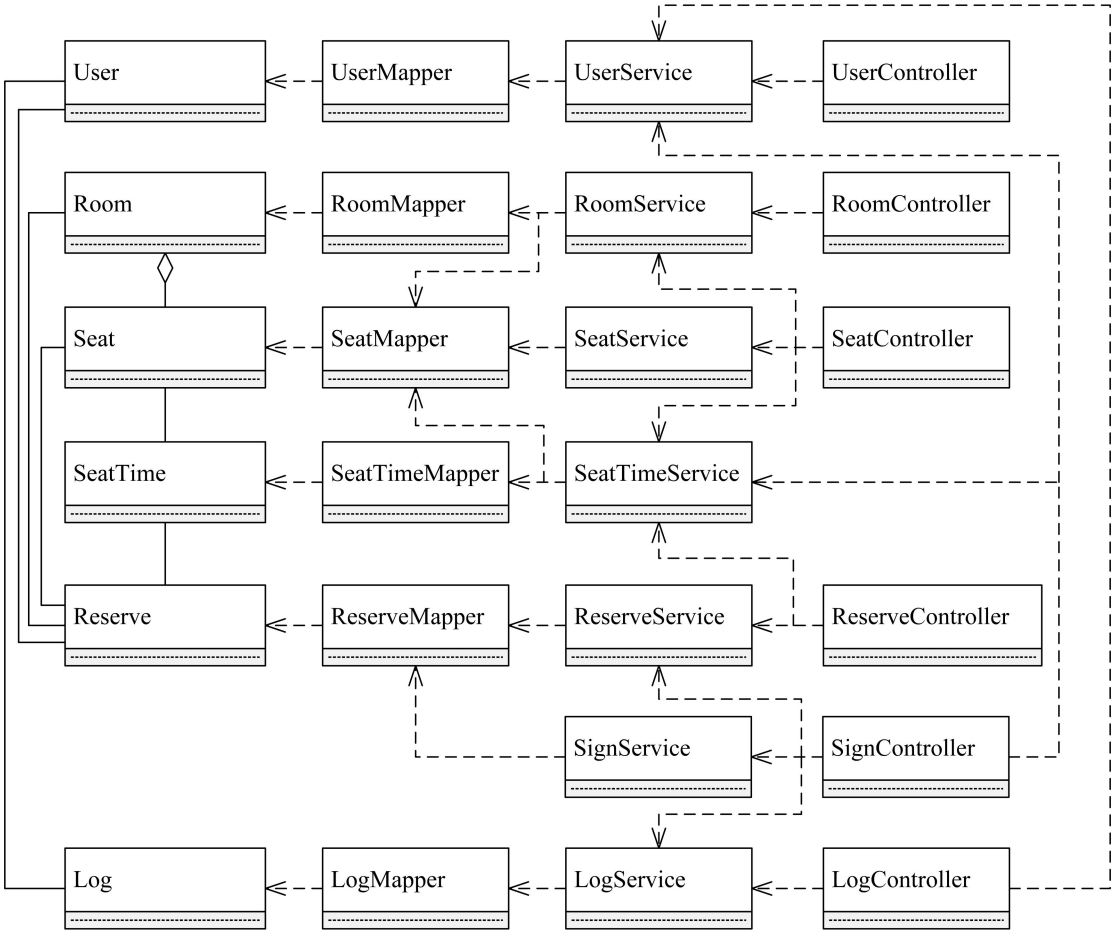


图 2 UML 类图

Mapper 与 Service 之间的关系如下所示：

(1) 依赖关系：RoomService 依赖 RoomMapper 和 SeatMapper。RoomService 类负责处理与自习室相关的业务逻辑，它依赖于 RoomMapper 来执行与自习室数据表相关的数据库操作，同时也依赖于 SeatMapper 来执行与座位数据表相关的操作，因为自习室与座位有业务关联。
(2) 依赖关系：SignService 依赖 ReserveMapper。SignService 类处理签到和签退的业务逻辑，它依赖于 ReserveMapper 来访问预约数据表，因为签到和签退操作需要查询或更新预约信息。
(3) 依赖关系：ReserveService 依赖 ReserveMapper。ReserveService 类管理预约的业务逻辑，它依赖于 ReserveMapper 来实现对预约数据表的 CRUD 操作。
(4) 依赖关系：SeatService 依赖 SeatMapper。SeatService 类处理与座位相关的业务逻辑，它依赖于 SeatMapper 来执行座位数据表的数据库操作。
(5) 依赖关系：SeatTimeService 依赖 SeatTimeMapper 和 SeatMapper。SeatTimeService 类负责管理座位使用时间的业务逻辑，它依赖于 SeatTimeMapper 来访问座位时间数据表，同时也可能依赖于 SeatMapper 来获取座位相关信息。
(6) 依赖关系：LogService 依赖 LogMapper。LogService 类处理日志记录的业务逻辑，它依

赖于 LogMapper 来存储和查询日志数据表中的记录。

(7) 依赖关系：UserService 依赖 UserMapper。UserService 类处理与用户相关的业务逻辑，包括用户认证、权限管理等，它依赖于 UserMapper 来访问用户数据表。

3.1.2 Entity 类

在本项目设计的智慧图书自习室管理系统中，共计设计了七个实体类，分别是：

(1) User (用户)，User 实体代表系统中的用户账户，包含用户 ID、账户、密码、姓名、权限等级、违规次数、违规状态和违规日期等属性。

(2) Room (房间)，Room 实体代表一个自习室，包含自习室 ID 和房间名称等属性。

(3) Reserve (预约)，Reserve 实体代表一个预约记录，包含预约 ID、用户账户、座位号、房间 ID、预约开始时间、预约结束时间、签到时间和签退时间以及预约状态等属性。

(4) Seat (座位)，Seat 实体代表房间内的单个座位，包含座位 ID、座位号和所属房间的 ID 等属性。

(5) SeatTime (座位时间)，SeatTime 实体代表座位的预约时间段，包含座位时间 ID、座位 ID、预约 ID、开始时间和结束时间等属性。

(6) Log (日志)，Log 实体代表系统日志，记录用户的操作历史，包含日志 ID、用户账户、日志状态和操作时间等属性。

(7) Book (图书)，Book 实体代表图书，记录图书的编号，书名，作者，出版社，借阅时间等信息。

3.1.3 Mapper 类

Mapper 层，即数据访问对象（DAO）层，它负责与数据库交互，执行 CRUD（创建、读取、更新、删除）操作。本次项目中，结合使用 SpringBoot 与 MyBatis-Plus 框架，将数据访问逻辑与业务逻辑分离，使得 Service 层可以专注于业务处理，而不必关心具体的数据库操作细节。其中，Spring 对 Mapper 接口进行容器管理，并通过依赖注入提供给 Service 层或其他需要数据访问的组件，MyBatis-Plus 作为 ORM（对象关系映射）框架，将 Java 对象映射到数据库表，以及将数据库查询结果映射到 Java 对象。

除此之外，MyBatis-Plus 提供了缓存机制和 SQL 优化功能，有助于提高数据访问层的性能，并且它支持动态 SQL，可以根据运行时的条件动态生成 SQL 语句，这在处理复杂查询时非常有用。

每个 Mapper 接口都继承自 BaseMapper，这是 MyBatis-Plus 提供的一个通用 Mapper 接口，它已经包含了许多基础的 CRUD 方法。由于继承了 BaseMapper，MyBatis-Plus 会在运行时自动实现这些接口的具体逻辑，所以不需要提供实现类。

3.1.4 Service 类

Service 层是应用程序架构中的一个核心层次，它负责实现应用程序的业务逻辑。在 Spring 框架中，Service 层通常由一系列 Spring 管理的组件（如@Service 注解的类）组成，这些组件负责处理应用程序的业务逻辑和事务管理。Service 层依赖于 Mapper 层（数据访问对象）来执行数据库的 CRUD 操作，或者 Service 层通过 Spring 的依赖注入机制使用其他 Service、Utility 类或 Mapper 接口。同时，Service 层的 Bean 通常被定义为单例（Singleton）作用域，因为业务逻辑通常不需要在多个实例间共享状态。除此之外，Service 层应该与表示层（如 Controller 层）松耦合，这样，如果表示层的实现发生变化，Service 层的业务逻辑可以保持不变。

Service 类的一般工作流程如下：

(1) 接收请求：Service 类接收来自 Controller 层的请求。
(2) 业务逻辑处理：Service 类实现具体的业务逻辑，包括数据验证、计算、业务规则应用等。
(3) 调用 Mapper 接口：Service 类通过调用 Mapper 接口的方法来与数据库交互，执行 CRUD 操作。
(4) 返回结果：Service 类处理 Mapper 接口返回的数据，并将其转换为业务层面需要的结果，然后返回给 Controller 层。

对应的变量与方法注解如下表所示。

表 1 Service 类变量与方法注解

Service 类名		变量名或方法名	描述
RoomService	依赖变量	roomMapper	用于操作房间数据表的 Mapper 接口
		seatMapper	用于操作座位数据表的 Mapper 接口
	方法	selectRoomId	根据房间 ID 查询房间信息
		selectRoomName	查找房间信息
		insertRoom	插入一个新的房间记录
		deleteRoom	删除一个房间记录及其所有座位
		updateRoom	更新房间信息
		selectRoomListById	根据房间 ID 查询房间列表
		selectRoomListByName	根据房间名查询房间列表
		selectAllList	查询所有房间信息
SignService	依赖变量	reserveMapper	用于操作预约数据表的 Mapper 接口
	方法名	signIn	处理用户的签到逻辑
		signOut	处理用户的签退逻辑
		selectSignInDelay	查找超时未签到的预约
		selectSignOutDelay	查找超时未签退的预约
ReserveService	依赖变量	reserveMapper	用于操作预约数据表的 Mapper 接口
	方法名	insertReserve	插入一个新的预约记录
		updateReserve	更新现有的预约记录
		deleteReserve	删除一个预约记录
		selectList	查询用户预约信息
		selectAllList	查询所有预约信息
SeatService	依赖变量	seatMapper	用于操作座位数据表的 Mapper 接口
	方法名	selectRoomId	根据座位查询所属房间的 ID
		insertSeat	插入一个新的座位记录
		deleteSeat	删除一个座位记录

		updateSeat	更新座位信息
		selectSeatList	根据座位号查询座位列表
		selectAllList	查询所有座位信息
SeatTimeService	依赖变量	seatTimeMapper	用于操作座位时间数据表的Mapper接口
		seatMapper	用于操作座位数据表的 Mapper 接口
	方法名	checkSeat	检查预约时间是否与现有预约冲突
		occupySeat	如果时间不冲突，则为预约分配座位
		findSeat	根据预约信息查找对应的座位
		releaseSeat	释放预约的座位
		deleteSeat	删除预约对应的座位时间
		updateSeatReserve	更新座位时间关联的预约 ID
		findSeatTime	查找预约对应的座位时间
		selectAllList	查询所有座位时间信息
LogService	依赖变量	logMapper	用于操作日志数据表的 Mapper 接口
	方法名	insertLog	插入一个新的日志记录
		updateLog	更新日志信息
		deleteLog	删除一个日志记录
		selectList	根据用户账号查询日志列表
		selectAllList	查询所有日志信息
UserService	依赖变量	userMapper	用于操作用户数据表的 Mapper 接口
	方法名	selectUserName	根据用户名查询用户
		insertUser	插入一个新的用户记录
		login	用户登录，验证用户账号和密码
		updateUser	更新用户信息
		deleteUser	删除一个用户记录
		selectUserList	根据用户账号查询用户列表
		selectAllList	查询所有用户信息
		selectUserAccount	根据用户账号查询用户信息
		addIllegal	添加违规状态，可能禁用用户
BookService	依赖变量	BookMapper	用于操作图书表的 Mapper 接口
	方法名	updateBook	更新图书信息
		deleteBook	删除图书记录
		selectBook	查询图书
		selectBookList	查询所有图书信息

3.1.5 Controller 类

Controller 层是 Spring MVC 架构中的一个关键组件，它负责处理用户的 HTTP 请求和返回响应。在本次 SpringBoot 应用程序中，Controller 层由带有注解的类和方法组成，用于定义应用程序的 API 端点。

Controller 层包含一个或多个方法，这些方法通过注解（如@RequestMapping 或@PostMapping）与特定的 HTTP 请求路径和请求类型（GET、POST、PUT、DELETE 等）关联。并且 Controller 方法通常委托 Service 层来执行业务逻辑，然后根据业务逻辑的结果构造响应。本次在设计 Controller 层中不同类时遵循 REST 原则，即使用 HTTP 方法和状态码来处理请求和响应。

Controller 类的工作流程如下：

(1) 接收请求：从客户端接收 HTTP 请求，通常以 JSON 格式的请求体或请求参数的形式。
(2) 处理业务逻辑：调用 Service 层的方法来处理业务逻辑。
(3) 返回响应：将处理结果封装成 HTTP 响应返回给客户端。

本次项目的 Controller 类层 UML 类图如下所示。

对应的方法注解与依赖的 Service 类如下表所示。

表 2 Controller 类方法注解

Controller 类名	方法名	依赖 Service 类	描述
SignController	signIn	SignService	处理用户的签到操作
	signOut	SignService	处理用户的签退操作
	autoSignInCancel	SignService, SeatTimeService	自动取消超时未签到的预约
	autoSignOutCancel	SignService, SeatTimeService	自动处理超时未签退的预约
ReserveController	reserve	ReserveService, SeatTimeService	新增预约信息
	update	ReserveService, SeatTimeService	更新预约信息
	delete	ReserveService	删除预约信息
	list	ReserveService	获取用户预约列表或所有预约列表
LogController	insertLog	LogService, UserService	插入日志记录
	updateLog	LogService	更新日志记录
	deleteLog	LogService	删除日志记录
	listAll	LogService	获取所有日志列表或用户账号对应的日志列表
SeatController	checkSeatOccupy	SeatTimeService, SeatService, RoomService	检查某个座位是否空闲
	insert	SeatService	添加座位信息
	update	SeatService	更新座位信息

	delete	SeatService	删除座位信息
	list	SeatService	获取座位列表或所有座位列表
UserController	register	UserService	用户注册
	login	UserService	用户登录
	update	UserService	更新用户信息
	delete	UserService	删除用户信息
	list	UserService	获取用户列表或所有用户列表
RoomController	insert	RoomService	添加房间信息
	update	RoomService	更新房间信息
	delete	RoomService	删除房间信息
	list	RoomService	根据房间 ID 或名称获取房间列表
BookController	insert	BookService	添加图书信息
	update	BookService	更新图书信息
	delete	BookService	删除图书信息
	list	BookService	根据图书 ID 或名称获取图书列表
	borrowBook	BookService	借阅图书
	returnBook	BookService	归还图书

最后，将包含实体类在内的四层框架组合起来，便形成了完整的后端构成，为智慧图书自习室预约管理系统的实现与完善夯实了框架与基础，完整的 UML 类图如下所示。

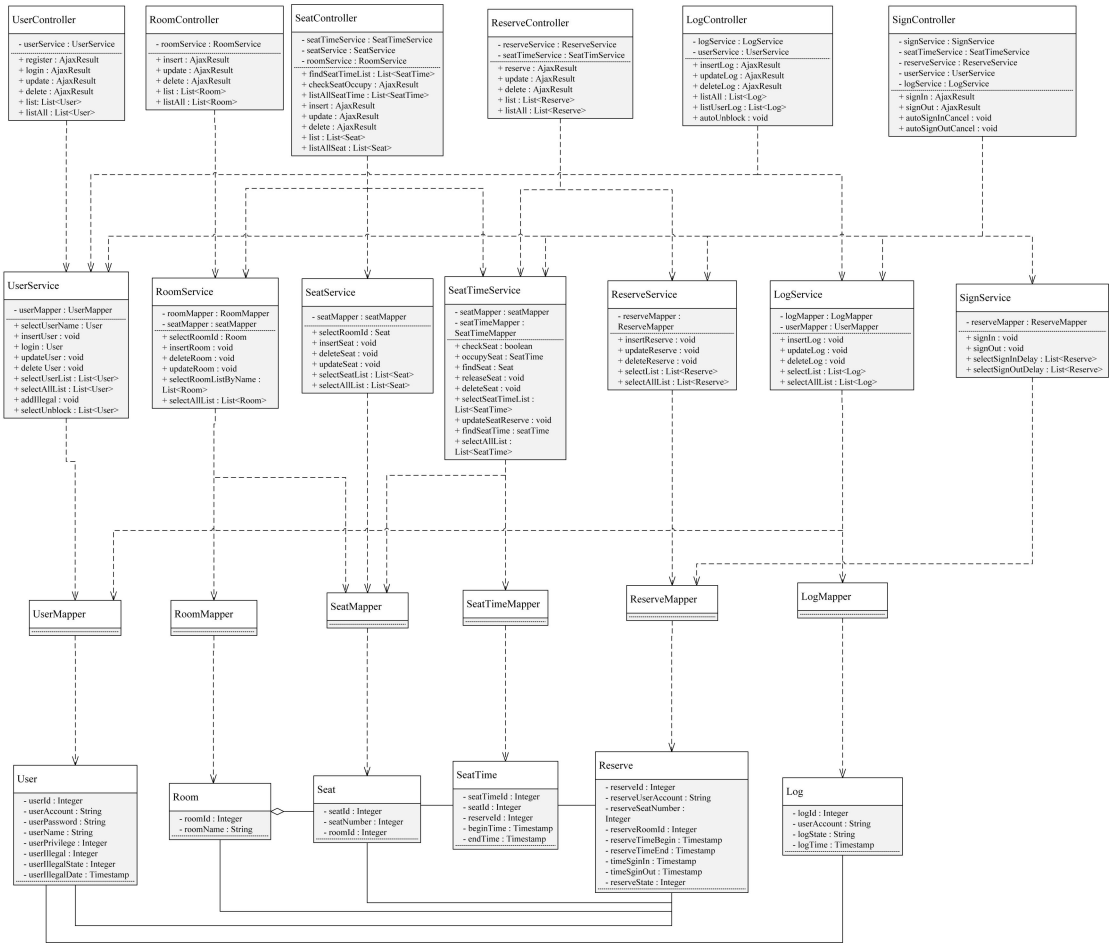


图 3 详细 UML 类图

3.2 接口设计

在接口设计方面，我们根据接口的相关主体将所有接口分成了五类，用户接口，预约接口，图书接口，座位接口和违规接口。我们还使用了小幺鸡文档管理工具，使得前端工作人员调用后端接口时可以更加清楚明了。

3.2.1 用户接口设计

表 3 用户接口-查找用户

接口名	查找用户		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/user/list?userAccount=zrj2003		
请求体	无		
接口描述	管理员进行查询某个用户信息的操作		
响应数据	userId：4 userAccount：zrj2003 userPassword：123456 userName：zrj2 userPrivilege：1 userIllegal：0 userIllegalState：0 userIllegalDate：null		

此接口主要实现功能是根据用户账号查询用户所有信息，由于 axios 自带的 GET 函数不能通过 request-body 来传输数据，所以我们将要传输的数据直接塞入 URL 中，再通过数组接收后端相应数据。

表 4 用户接口-查询所有用户

接口名	查询所有用户		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/user/all		
请求体	无		
接口描述	将所有用户信息返回前端，可在管理员界面进行查看。		
响应数据	userId：4 userAccount：zrj2003 userPassword：123456 userName：zrj2 userPrivilege：1 userIllegal：0 userIllegalState：0 userIllegalDate：null		

	userId：5 userAccount： zrj2005 userPassword： 123456 userName： zrj3 userPrivilege：0 userIllegal：0 userIllegalState：0 userIllegalDate：null

此接口实现的功能是返回所有用户的信息，主要应用在管理员界面的用户管理分支，方便管理员对用户信息进行查看，返回的数据包括用户 Id，用户账号，用户密码，用户名，用户权限，用户违规次数，当前是否违规，封禁日期。

表 5 用户接口-删除用户

接口名	删除用户		
请求方法	DELETE	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/user/delete		
请求体	userId：1		
接口描述	管理员进行删除指定用户的操作		
响应数据	msg： 操作成功 code：200		

此接口实现的功能是删除指定用户，根据前端传入的用户 Id 找到对应用户并进行删除操作，主要应用在店铺值班人员发现某用户行为极为恶劣的情况，管理员可以直接删除该账号。

表 6 用户接口-修改用户信息

接口名	修改用户信息		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/user/update		
请求体	userId：4 userName：zrj userPassword：123456 userPrivilege :0		
接口描述	管理员进行修改指定用户的用户名，密码和权限的操作		
响应数据	操作成功返回： msg： 操作成功		

	code：200 用户名重复返回： msg： 用户名重复 code：500

此接口实现的功能是管理员在管理员界面修改指定用户的信息，前端可选择传入该用户用户名，密码，权据，后端只根据用户 Id 修改前端传入的数据，如果未传入则保持原先数据值。主要应用在可以修改账号权限，新增或减少管理员数量；如果用户忘记密码也可以委托管理员进行修改。

表 7 用户接口-用户登录

接口名	用户登录		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/user/login		
请求体	userAccount： zrj2003 userPassword： 123456		
接口描述	用户进行登录操作		
响应数据	登录成功返回： msg： 操作成功 code：200 token：0 账户或密码错误返回： msg： 用户名或密码错误 code：500		

此接口实现的功能是实现用户登录操作，前端传入用户的账号和密码，如果和数据库中内容匹配则进行鉴权操作，决定登录预约界面还是管理员界面，否则显示账号或密码错误。

表 8 用户接口-用户注册

接口名	用户注册		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/user/register		
请求体	userAccount： zrj2002 userName： zrj userPassword： 123456		
接口描述	用户进行注册操作		
响应数据	注册成功返回：		

	msg： 操作成功 code：200 用户名或账户重复返回： msg： 用户名或账户重复 code：500

此接口实现的功能是实现用户注册操作，前端传入用户的账号、用户名和密码，如果账号和用户名未重复则数据库记录该数值，否则提示用户“用户名或账号重复”，之后迭代会加入验证码模块，记录用户注册使用的手机号，方便封禁账号的操作。

3.2.2 预约接口设计

表 9 预约接口-新增预约

接口名	新增预约		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/reserve/add		
请求体	reserveUserName： zrj2004 reserveSeatNumber：2 reserveRoomId：2 reserveTimeBegin： 2024-04-01 15:45:16 reserveTimeEnd： 2024-04-01 17:45:15		
接口描述	用户或管理员进行预约的操作		
响应数据	预约成功返回： msg： 操作成功 code：200 座位被占用返回： msg： 座位已被占用 code：500		

此接口实现的功能是管理员或用户可以进行预约，主要应用在用户进入系统预约图书馆或用户委托管理员进行预约操作，预约时前端传入预约账号，预约座位号，预约房间号，预约开始结束时间，如果当前选择的座位被占用则提示用户重新选择座位。

表 10 预约接口-修改预约

接口名	修改预约		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/reserve/update		
请求体			

	reserveId :19 reserveUserName： zrj2004 reserveSeatNumber：2 reserveRoomId：2 reserveTimeBegin： 2024-04-01 15:45:16 reserveTimeEnd： 2024-04-01 17:46:15
接口描述	管理员或用户进行修改预约信息的操作
响应数据	修改成功返回： msg： 操作成功 code：200 座位被占用返回： msg： 座位已被占用 code：500

此接口实现的功能是管理员或用户可以修改预约信息，后端根据预约 Id 修改前端传入的数据。主要应用在用户预约后希望调整预约时间或位置，用户委托管理员修改预约时间或位置。

表 11 预约接口-查询预约

接口名	查询预约		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/reserve/list?userAccount=zrj2004		
请求体	无		
接口描述	管理员或用户进行查询该账户预约信息的操作		
响应数据	reserveId：18 reserveUserAccount： zrj2004 reserveSeatNumber：2 reserveRoomId：2 reserveTimeBegin： 2024-04-01 17:45:16 reserveTimeEnd： 2024-04-01 18:44:16 timeSignIn：null timeSignOut： 2024-04-15 08:44:40 reserveState：1		

此接口实现的功能是管理员或用户可以查询该账号所有预约信息，由于 axios 自带的 GET 函数不能通过 request-body 来传输数据，所以我们将要传输的数据直接塞入 URL 中，再通过数组接收后端传回的数据，包括预约 Id，预约账号，预约座位，预约房间号，预约开始结束时间，签到签退时间，当前预约信息状态(1-未签退，2-自动签退，3-手动签退)。

表 12 预约接口-查询所有预约

接口名	查询所有预约		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/reserve/all		
请求体	无		
接口描述	将所有预约信息返回前端，可在管理员界面进行查看。		
响应数据	reserveId：18 reserveUserAccount： gly2003 reserveSeatNumber：2 reserveRoomId：2 reserveTimeBegin： 2024-04-01 17:45:16 reserveTimeEnd： 2024-04-01 18:44:16 timeSignIn：null timeSignOut： 2024-04-15 08:44:40 reserveState：1 reserveId：19 reserveUserAccount： zrj2004 reserveSeatNumber：2 reserveRoomId：2 reserveTimeBegin： 2024-04-01 15:45:16 reserveTimeEnd： 2024-04-01 17:44:15 timeSignIn： 2024-04-03 15:45:16 timeSignOut： 2024-04-15 08:44:40 reserveState：1		

此接口实现的功能是返回所有用户的预约信息，主要应用在管理员界面的预约管理分支，方便管理员对用户预约信息进行查看，返回的数据包括预约 Id，预约账号，预约座位号，预约房间号，预约开始结束时间，签到签退时间和当前预约状态(1-未签退，2-自动签退，3-手动签退)。

表 13 预约接口-取消预约

接口名	取消预约		
请求方法	DELETE	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/reserve/delete		
请求体	reserveId：18		
接口描述	用户或管理员进行取消预约操作		
响应数据	操作成功：		

	msg： 操作成功 code：200

此接口实现的功能是管理员或用户可以删除预约信息，后端根据预约 Id 删除对应的预约信息。主要应用在用户预约后日程改变希望删除预约；用户违规恶劣，被管理员手动取消预约；用户迟到超过 30 分钟被系统自动取消预约。

表 14 预约接口-用户签到

接口名	用户签到		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/sign/in		
请求体	reserveId：18 timeSignIn： 2024-04-15 08:44:40		
接口描述	用户进行签到操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是用户可以进行签到操作，前端传入预约 Id 和用户签到的当前时间，后端将其存入数据库中。

表 15 预约接口-用户签退

接口名	用户签退		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/sign/out		
请求体	reserveId：18 timeSignOut： 2024-04-15 09:44:40		
接口描述	用户进行签退操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是用户可以进行签退操作，前端传入预约 Id 和用户签退的当前时间，后端将其存入数据库中。

3.2.3 座位接口设计

表 16 座位接口-新增座位

接口名	新增座位		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/seat/insert		

请求体	seatNumber :7 roomId :3
接口描述	管理员进行添加座位操作
响应数据	操作成功： msg： 操作成功 code：200 房间号存在，但座位号重复 msg： 座位号重复 code：500 自习室不存在 msg： 自习室不存在 code：501

此接口实现的功能是管理员可以添加新的座位，主要应用在线下自习室添加新的座位后管理员在数据库中可以添加相对应的座位信息。前端传入该座位在自习室的编号和所属自习室的 Id，如果自习室不存在或座位编号重复则提示管理员。

表 17 座位接口-修改座位

接口名	修改座位		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/seat/update		
请求体	seatId :7 seatNumber :7 roomId :3		
接口描述	管理员进行修改座位信息操作		
响应数据	操作成功： msg： 操作成功 code：200 房间号存在，但座位号重复 msg： 座位号重复 code：500 自习室不存在		

	msg： 自习室不存在 code：501

此接口实现的功能是管理员可以修改座位信息，主要应用在线下自习室座位变动后，管理员在数据库中可以修改相对应的座位。前端传入该座位 Id、所在图书馆的编号和所属图书馆的 Id，如果自习室不存在或座位编号重复则提示管理员。

表 18 座位接口-删除座位

接口名	删除座位		
请求方法	DELETE	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/seat/delete		
请求体	seatId :7		
接口描述	管理员进行删除座位操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是管理员可以删除座位，主要应用在线下自习室座位损坏后，管理员在数据库中可以删除相对应的座位。前端传入该座位 Id，后端接收后删除数据库对应座位。

表 19 座位接口-查找所有座位

接口名	查找所有座位		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/seat/allseat		
请求体	无		
接口描述	将所有座位信息返回前端，可在管理员界面进行查看。		
响应数据	seatId：1 seatNumber：1 roomId：1 seatId：2 seatNumber：2 roomId：1 seatId：3 seatNumber：1 roomId：2		

此接口实现的功能是返回所有座位信息，主要应用在管理员界面的座位管理分支，方便管理员对座位信息进行查看，返回的数据包括座位 Id，座位在图书馆中的编号，所属图书馆的 Id。

表 20 座位接口-查询座位占用时间表

接口名	查询座位占用时间表		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/seat/list?seatId=2		
请求体	无		
接口描述	将该座位被占用时间返回前端，方便根据座位占用情况改变预约界面座位颜色。		
响应数据	seatTimeId：32 seatId：2 reserveId：27 beginTime： 2024-04-07 02:46:00 endTime： 2024-04-07 03:46:00 seatTimeId：36 seatId：2 reserveId：31 beginTime： 2024-04-08 01:03:00 endTime： 2024-04-08 04:03:00		

此接口实现的功能是返回指定座位被占用的时间，主要应用在前端根据座位占用情况改变预约界面座位颜色，返回的数据包括座位占用 Id，座位 Id，预约 Id，被占用开始结束时间。

表 21 座位接口-查询所有座位占用时间表

接口名	查询所有座位占用时间表		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/seat/allseattime		
请求体	无		
接口描述	将所有座位被占用时间返回前端，可在管理员界面进行查看。		
响应数据	seatTimeId：32 seatId：2 reserveId：27 beginTime： 2024-04-07 02:46:00 endTime： 2024-04-07 03:46:00		

	seatTimeId：36 seatId：4 reserveId：31 beginTime： 2024-04-08 01:03:00 endTime： 2024-04-08 04:03:00

此接口实现的功能是返回所有座位被占用的时间，主要应用在前端根据座位占用情况改变预约界面座位颜色，返回的数据包括座位占用 Id，座位 Id，预约 Id，被占用开始结束时间。

3.2.4 自习室接口设计

表 22 自习室接口-新增自习室

接口名	新增自习室		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/insert		
请求体	roomName :room6		
接口描述	管理员进行添加自习室操作		
响应数据	操作成功： msg： 操作成功 code：200 自习室已存在 msg： 自习室已存在 code：500		

此接口实现的功能是管理员可以添加新的自习室，主要应用在线下自习室添加新的房间后管理员在数据库中可以添加相对应的房间信息。前端传入该座位在自习室名，如果自习室名重复则提示管理员。

表 23 自习室接口-修改自习室

接口名	修改自习室		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/update		
请求体	roomId：2 roomName :room6		
接口描述	管理员进行修改自习室信息操作		
响应数据	操作成功：		

	msg： 操作成功 code：200 自习室名重复 msg： 自习室名重复 code：500

此接口实现的功能是管理员可以修改自习室信息，主要应用在线下自习室变动后，管理员在数据库中可以修改相对应的自习室信息。前端传入该自习室 Id 和自习室名，如果自习室名重复则提示管理员。

表 24 自习室接口-删除自习室

接口名	删除自习室		
请求方法	DELETE	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/delete		
请求体	roomId：2		
接口描述	管理员进行删除自习室信息操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是管理员可以删除自习室信息，主要应用在线下自习室变动后，管理员在数据库中可以删除相对应的自习室信息。前端传入该自习室 Id，后端根据该 Id 删除数据库中对应自习室。

表 25 自习室接口-查找所有自习室

接口名	查找所有自习室		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/all		
请求体	无		
接口描述	将所有自习室信息返回前端，可在管理员界面进行查看。		
响应数据	roomId：2 roomName :room2 roomId：3 roomName :room3		

	roomId：4 roomName :room4

此接口实现的功能是返回所有自习室信息，主要应用在管理员界面的自习室管理分支，方便管理员对自习室信息进行查看，返回的数据包括自习室 Id，自习室名。

表 26 自习室接口-根据 Id 查找自习室

接口名	根据 roomId 查找自习室		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/listbyid?roomId=5		
请求体	无		
接口描述	自习室根据 roomId 进行查询自习室操作		
响应数据	roomId：2 roomName :room2		

此接口主要实现功能是根据自习室 Id 查询该自习室的信息，由于 axios 自带的 GET 函数不能通过 request-body 来传输数据，所以我们将要传输的数据直接塞入 URL 中，再通过数组接收后端相应数据。

表 27 自习室接口-根据 roomName 查找自习室

接口名	根据 roomName 查找自习室		
请求方法	GET	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/room/listbyid?roomName=room5		
请求体	无		
接口描述	管理员根据 roomName 进行查询自习室操作		
响应数据	roomId：5 roomName：room5		

此接口主要实现功能是根据自习室名查询该自习室的信息，由于 axios 自带的 GET 函数不能通过 request-body 来传输数据，所以我们将要传输的数据直接塞入 URL 中，再通过数组接收后端相应数据。

3.2.5 违规接口设计

表 28 违规接口-新增违规信息

接口名	新增违规信息		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/log/insert		
请求体	userAccount： gly2003		

	logState：大声喧哗
接口描述	管理员根据用户实际违规情况，添加违规信息操作
响应数据	操作成功： msg： 操作成功 code：200

此接口实现的功能是管理员可以添加新的违规信息，前端传入用户账号和违规详情，主要应用在用户在自习期间做出违规行为，管理员可以将其记录；当用户预约了但是迟到 30 分钟未签到，系统会自动取消该预约并在违规数据库中记录该用户的迟到行为。

表 29 违规接口-修改违规信息

接口名	修改违规信息		
请求方法	PUT	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/log/update		
请求体	logId :1 userAccount： gly2003 logState：未及时签到 logTime：2024-04-12 02:46:00		
接口描述	管理员根据用户实际违规情况，修改违规信息操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是管理员可以修改违规信息，前端传入该违规 Id、违规账号，违规详情，封禁时间，主要应用在用户被封禁后申诉，希望管理员调整封禁时间。

表 30 违规接口-删除违规信息

接口名	删除违规信息		
请求方法	DELETE	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/log/delete		
请求体	logId :1		
接口描述	管理员根据用户实际违规情况，删除违规信息操作		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是管理员可以删除违规信息，前端传入该违规 Id、违规账号，违规详情，封禁时间，主要应用在用户被封禁后申诉，希望管理员可以删除该违规。

表 31 违规接口-查询所有违规信息

接口名	查询所有违规信息		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/log/all		
请求体	无		
接口描述	将所有违规信息返回前端，可在管理员界面进行查看。		
响应数据	logId：1 userAccount： zrj2002 logState： 迟到 30 分钟 logTime： 2024-04-27 13:57:50 logId：2 userAccount： gly2003 logState： 大声喧哗 logTime： 2024-04-27 13:28:40		

此接口实现的功能是返回所有用户的违规信息，主要应用在管理员界面的违规管理分支，方便管理员对用户违规信息进行查看，返回的数据包括违规 Id，违规账号，违规详情，封禁时间。

表 32 违规接口-查找违规信息

接口名	查询违规信息		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/log/select?userAccount=zrj2004		
请求体	无		
接口描述	将该账户的所有违规信息返回前端，可在管理员界面进行查看。		
响应数据	logId：1 userAccount： zrj2002 logState： 迟到 30 分钟 logTime： 2024-04-27 13:57:50 logId：2 userAccount： zrj2002 logState： 大声喧哗 logTime： 2024-04-27 13:28:40		

此接口主要实现功能是根据用户账号查询用户所有违规信息，由于 axios 自带的 GET

函数不能通过 request-body 来传输数据，所以我们将要传输的数据直接塞入 URL 中，再通过数组接收后端相应数据。

3.2.6 图书接口设计

表 33 图书接口-查询图书

接口名	查询图书		
请求方法	GET	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/book/booklist		
请求体	无		
接口描述	将所有图书信息返回前端，可在管理员界面进行查看。		
响应数据	bookId：3 number ： A02 bookName ： 论 gly2 author ： 赵瑞杰 publisher ： 机械工业出版社 serialNumber ： 0000-0001-0002 borrowState ：0 borrowUserId ：null borrowDate ：null returnDate ：null		

此接口实现的功能是返回所有图书信息，主要应用在管理员界面的图书管理分支，方便管理员对图书信息进行查看，返回的数据包括图书 Id，图书位置编号，图书名，作者，出版社，ISBN 号，借阅人信息。

表 34 图书接口-删除图书

接口名	删除图书		
请求方法	DELETE	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/book/delete		
请求体	bookId :1		
接口描述	按照 bookId 将书本删除。		
响应数据	操作成功： msg： 操作成功 code：200		

此接口实现的功能是管理员可以删除图书信息，前端传入该图书的图书 Id，后端根据该图书 Id 删除该图书。

表 35 修改图书信息

接口名	修改图书信息

请求方法	PUT	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/book/update		
请求体	bookId :1 number :A02 bookName :图书 1 author :赵瑞杰 publisher :宁波大学出版社 serialNumber :1000-0000-2222		
接口描述	修改图书信息。		
响应数据	操作成功： msg ： 操作成功 code ： 200		

此接口实现的功能是管理员可以修改图书信息，前端传入该图书 Id、图书位置编号，图书名，作者，出版社，ISBN 号。

表 36 图书接口-添加图书信息

接口名	添加图书信息		
请求方法	POST	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/book/insert		
请求体	number :A02 bookName :图书 1 author :赵瑞杰 publisher :宁波大学出版社 serialNumber :1000-0000-2222		
接口描述	添加图书信息。		
响应数据	操作成功： msg ： 操作成功 code ： 200		

此接口实现的功能是管理员可以添加新的图书规信息，前端传入图书相关信息，使用 POST 方法将图书信息存入到数据库中。

表 37 图书接口-借书

接口名	借书		
请求方法	POST	请求数据类型	JSON
接口 URL	http://124.71.194.156:8080/book/borrow		
请求体	bookId :2 bookUserId :1		

	borrowDate :2024-04-03 15:45:19
接口描述	用户借阅图书。
响应数据	操作成功： msg： 操作成功 code：200

通过 POST 请求方式实现图书借阅功能，请求体包含图书 ID、借书用户 ID、借书时间和预计还书时间。正确请求后，服务器将返回 200 状态码和"操作成功"的响应消息，以确认借书操作正确执行。

表 38 图书接口-还书

接口名	借书		
请求方法	POST	**请求数据类型**	JSON
接口 URL	http://124.71.194.156:8080/book/return		
请求体	bookId :2 bookUserId :1 returnDate :2024-04-03 15:45:19		
接口描述	用户还书。		
响应数据	操作成功： msg： 操作成功 code：200		

通过 POST 请求方式实现还书功能，请求体包含图书 ID、借书用户 ID、还书时间。正确请求后，服务器将返回 200 状态码和"操作成功"的响应消息，以确认还书操作正确执行。

四、软件展示

4.1 登录界面

4.1.1 用户登录

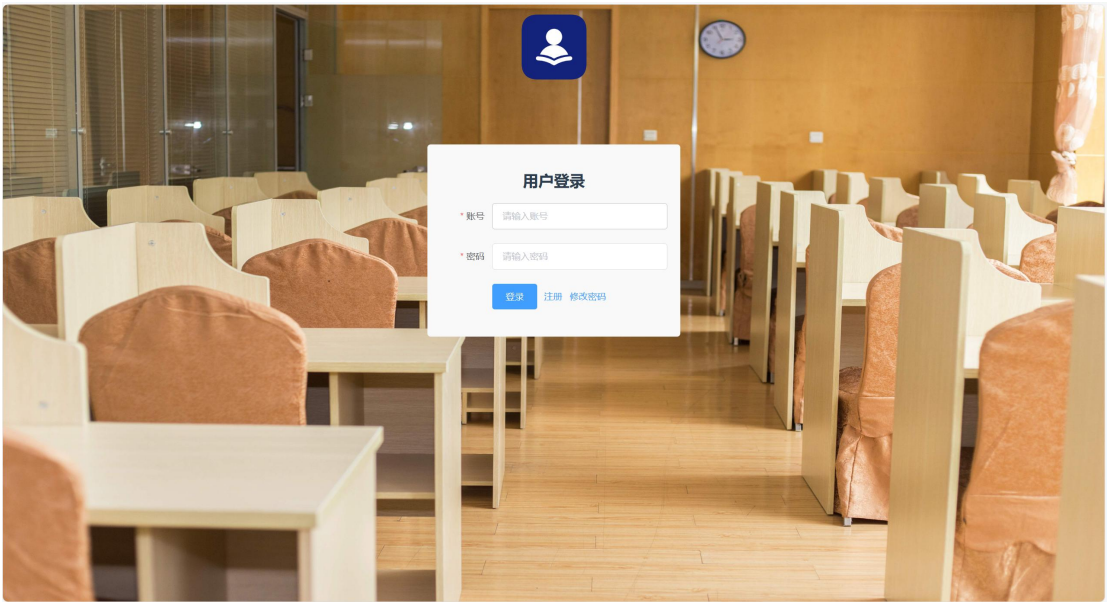


图 5 用户登录界面整体展示

有两个输入框分别用户输入账号和密码，如果未输入就点击登录按钮，则前端规则判断后在输入框下显示“请输入账号”，“请输入密码”。用户输入账号和密码点击登录按钮后，如果账号和密码匹配则登录成功，否则根据情况进行报错提示。

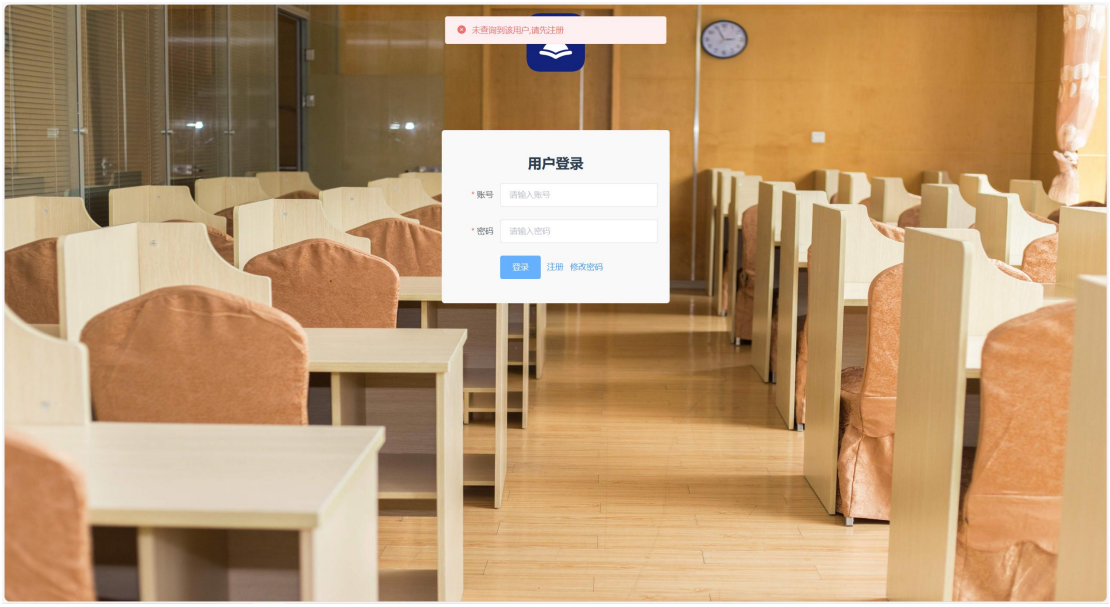


图 6 用户未注册提示

在用户登录的时候我们使用 axios 自带的 post 方法，将用户输入的账号和密码传回后端进行判断，如果未查询到该用户说明数据库中未包含该用户的数据则抛出报错“未查询到该用户，请先注册”来提示用户。

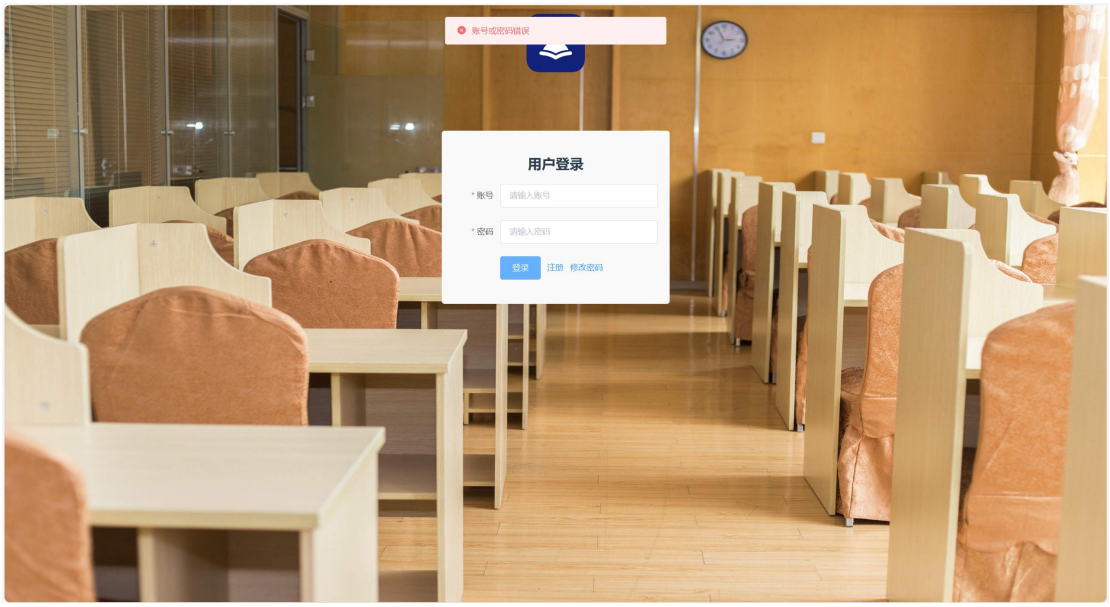


图 7 用户账号密码输入错误

将用户账号密码 post 到后端后，后端代码根据用户账号进行查询，查找到该用户后将传入的密码和数据库存储的密码相比较，如果比对错误则返回错误代码 500，前端界面接受到错误代码 500 后提示用户“账号或密码错误”，并清空账号输入框和密码输入框以此来提示用户重新输入账号和密码。

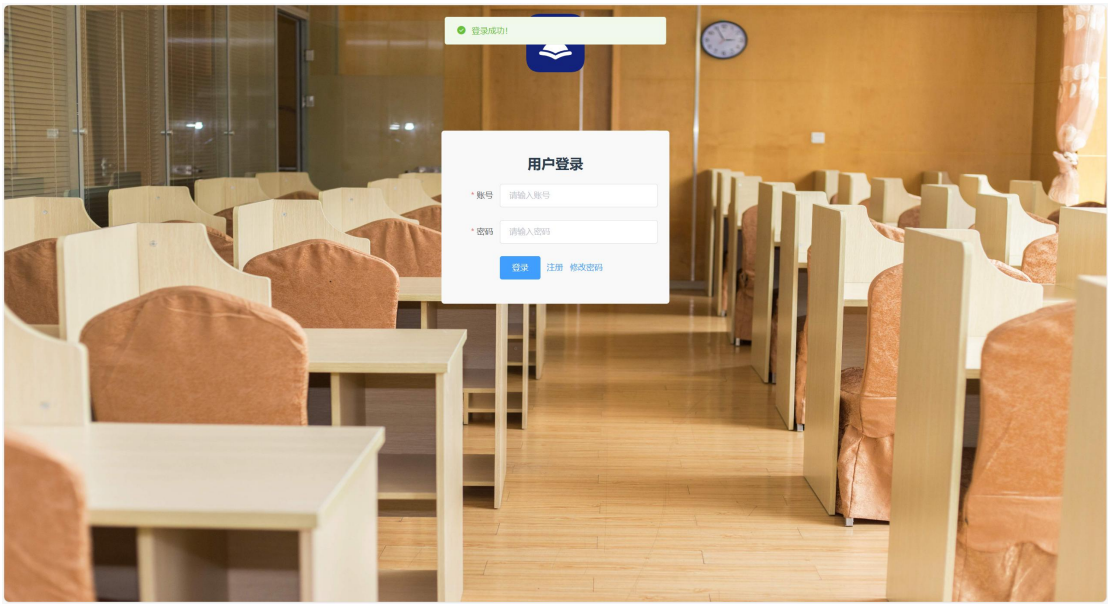


图 8 用户登录成功

将用户账号密码 post 到后端后，后端代码根据用户账号进行查询，查找到该用户后将传入的密码和数据库存储的密码相比较，如果比对成功则返回代码 200，前端界面接受到代码 200 后提示用户“登陆成功”。

4.1.2 用户注册

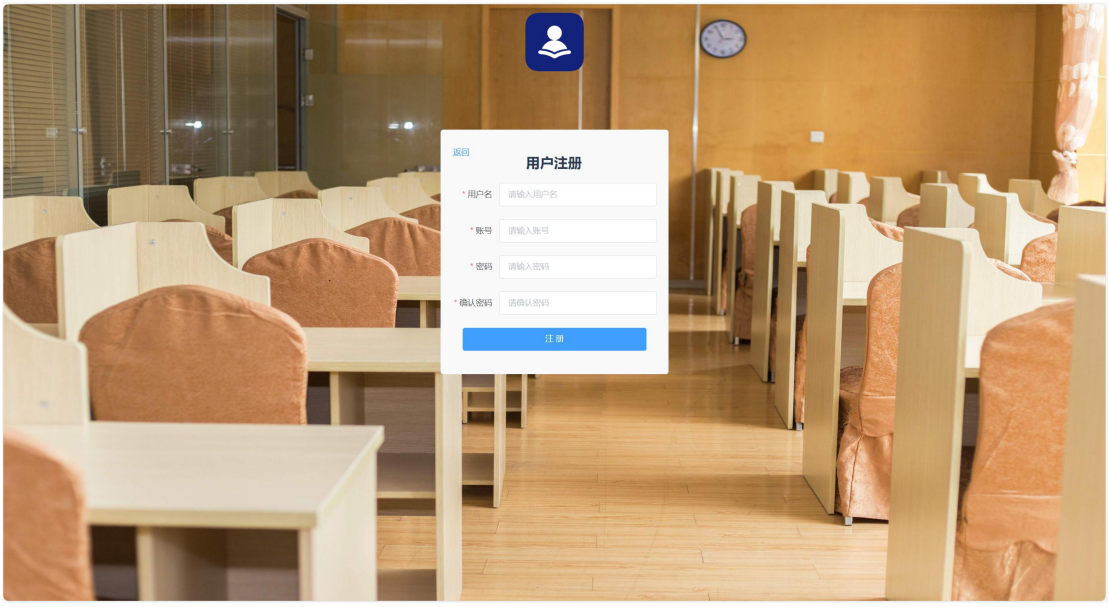


图 9 用户注册界面整体展示

用户输入用户名，账号，密码和确认密码进行注册操作，使用 post 函数将输入内容传入后端，后端判断该输入用户名和账号是否和数据库已存在的数据重复，如果重复则返回 500 报错，未重复则提示用户注册成功。

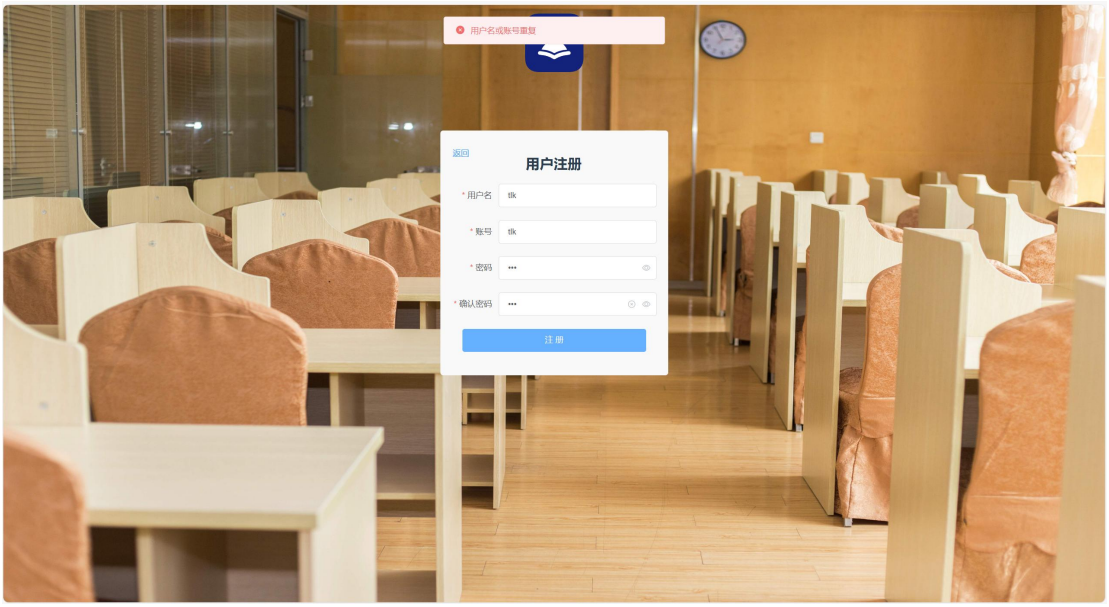


图 10 用户注册重复

后端读取前端传入的数据，并检索用户数据表，如果数据重复则返回错误代码 500，前端接受到错误代码 500 后显示“用户名或账号重复”，以此提示用户重新输入用户名和账号。

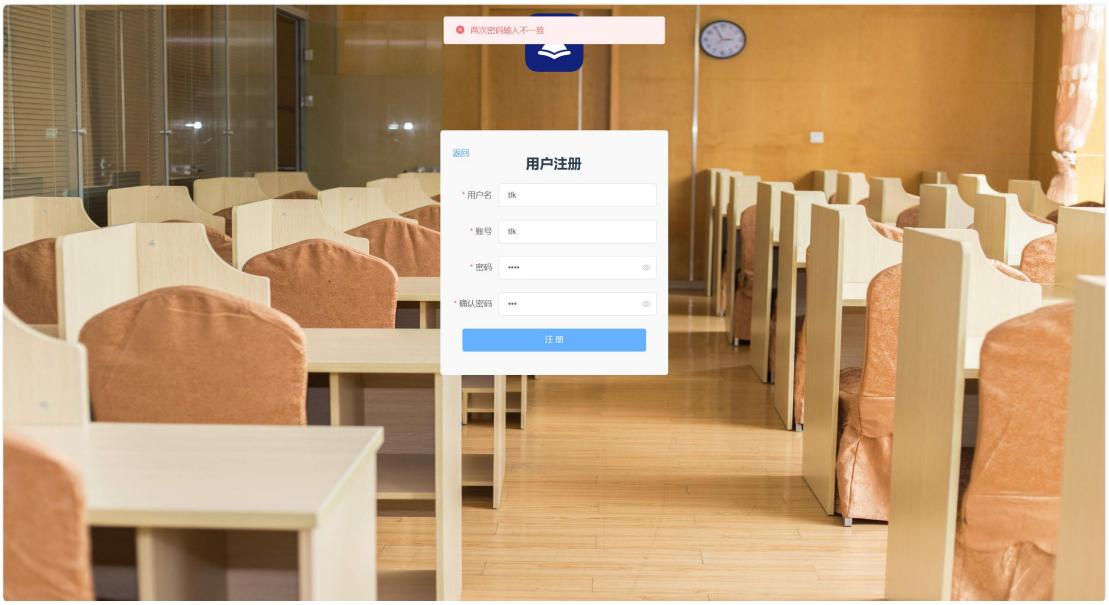


图 11 用户密码输入不一致

在前端界面判断用户输入的密码是否一致，如果一致再将数据 post 到后端进行存储，以免造成用户注册的错误。

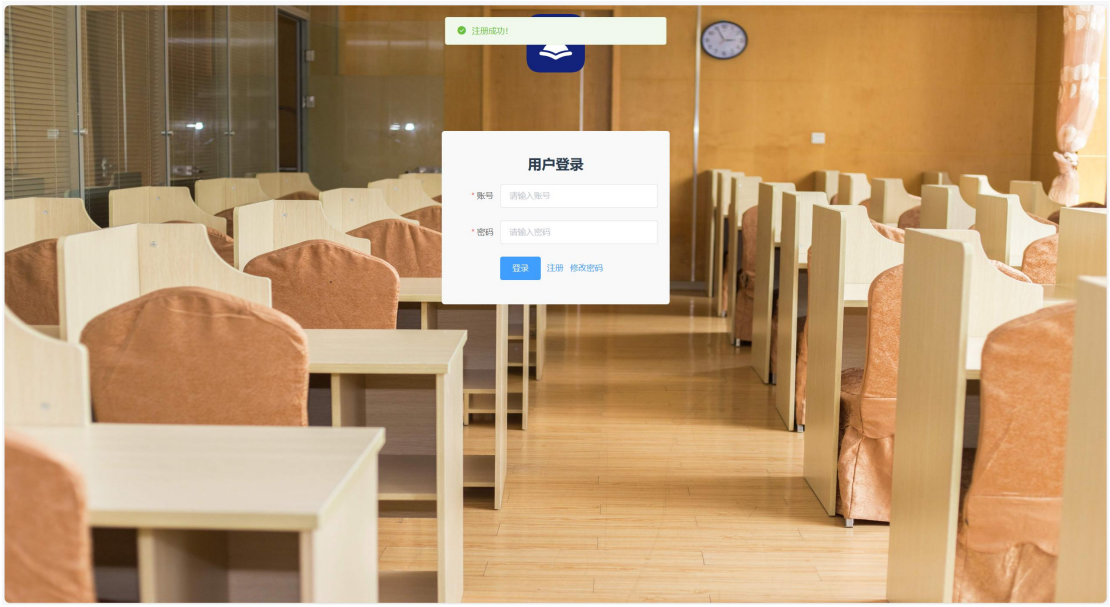


图 12 用户注册成功

经过后端检测该新注册的用户并未重复，就将数据存储到数据库用户表中，并自动跳转到登录界面，以便新注册的用户进行登录操作。

4.2 用户界面

4.2.1 自习室预约

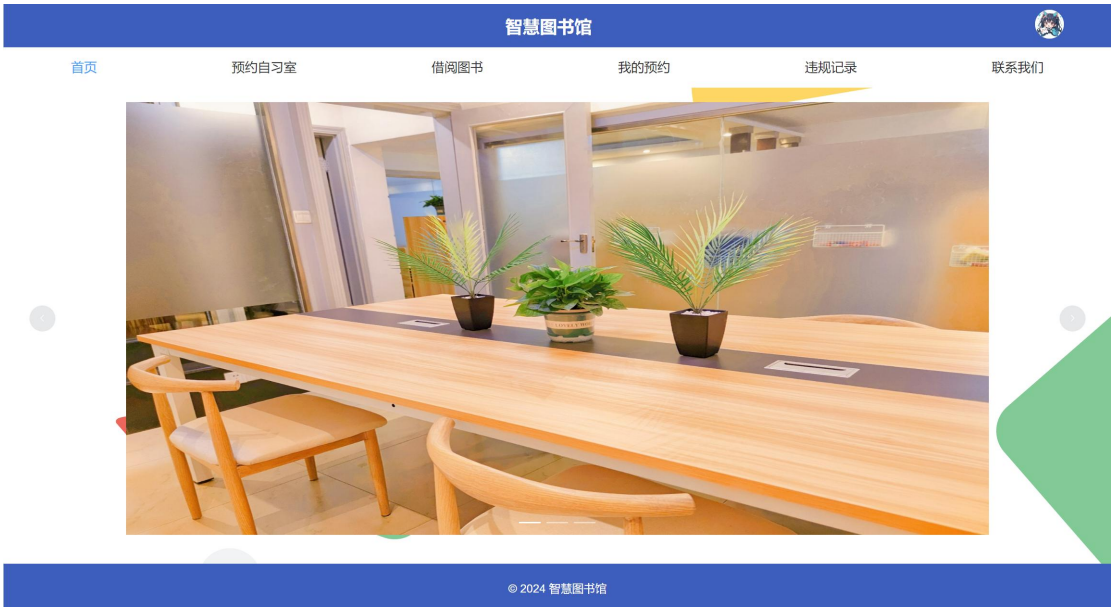


图 13 自习室展示界面

通过 axios 自带 get 函数读取数据库自习室表的数据，再在前端遍历显示自习室图片。



图 14 自习室详细信息界面

点击自习室下的“详细信息”按钮后，前端函数读取数据库座位表数据，根据自习室座位数量，动态设置该自习室中座位的展示界面，并将预约日期默认设置为当前日期前，预约起始时间设置为当前时间。



图 15 用户选择时间和座位

用户可以输入预约日期和预约起始结束时间并选择座位，可选的预约日期包括今天到后天三天可以选择，预约起始时间为当前时间到晚上 23 点，用户选择座位后会提示用户该座位是几号座位，该图展示的座位排列也与实际座位对应，用户可以选择自己心仪的座位。



图 16 座位部分占用提示

点击自习室“详细信息”或选择预约时间段后，如果该座位在当前时间到晚上 22：59 之间的部分时间或用户选择时间段内被部分占用，则该座位会显示黄色，提示“该座位在当前时间段和他人预约时间重合，请调整时间”，告知用户重新选择时间段或重新选择座位。



图 17 座位完全占用提示

点击自习室“详细信息”或选择预约时间段后，如果该座位在当前时间到晚上 22：59 之间的部分时间或用户选择时间段内被全部占用，则该座位会显示红色，提示“该时间段已被全部预约满，请选择其他座位”，告知用户重新选择时间段或重新选择座位。

4.2.2 预约信息查询



图 18 预约信息展示界面

用户预约过后可以在“我的预约”看到自己账号所有的预约信息，表单会显示预约的主要信息，包括预约时间段，自习室号，座位号。

4.2.3 签到签退

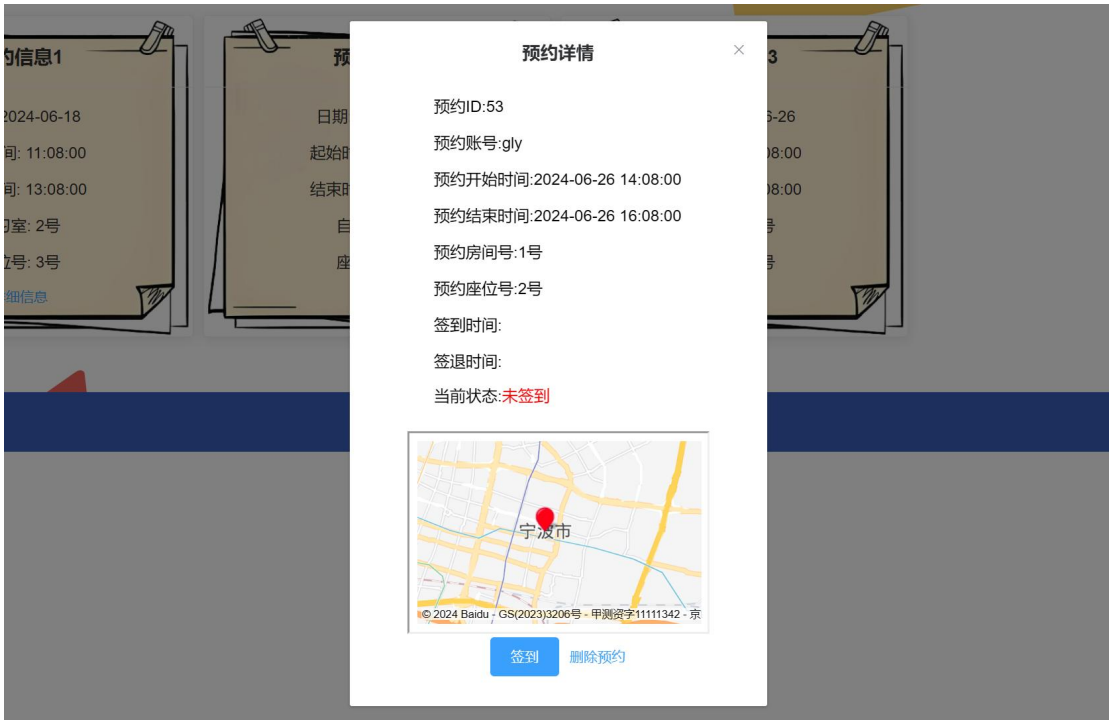


图 19 预约信息详情界面

点击详细信息后，用户可以查看更多预约信息细节，比如显示该预约的状态，我们将预约状态分为了“未开始”，“未签到”，“已签到”，“已签退”，“已过期”，“已自动签退”，“迟到 30 分钟以上，预约自动取消”这七种状态。

如果未到预约开始时间则显示“未开始”；到了预约开始时间则显示“未签到”；用户点击签到按钮成功签到后显示“已签到”；用户点击签退按钮成功签退后显示“已签退”；预约结束时间早于当前时间则显示“已过期”；如果用户在预约开始时间 30 分钟后还未签到，则系统自动取消该预约，并显示“迟到 30 分钟以上，预约自动取消”；如果用户签到了但是忘记签退，则在预约时间结束后 5 分钟，系统自动帮该用户进行签退操作，显示“已自动签退”

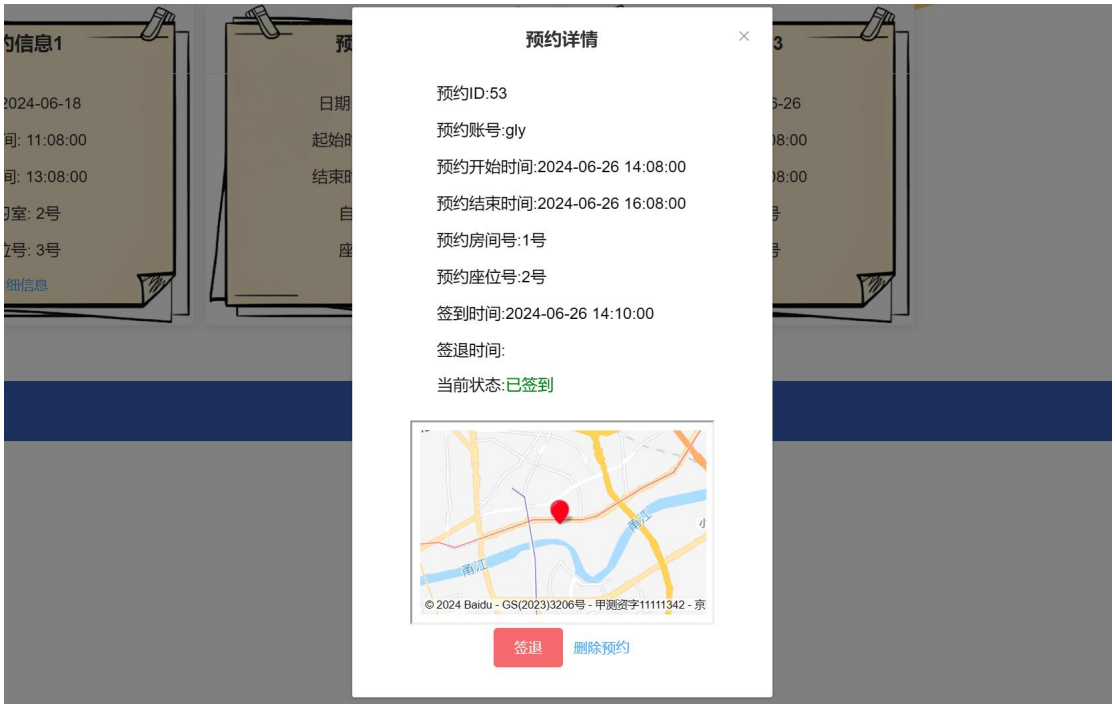


图 20 用户签到

如果该预约状态为“未签到”则在详细信息界面显示签到按钮，然后定位用户当前所在位置，如果用户当前在签到范围内则用户点击签到按钮后，用户签到时间被置为当前时间，并将该时间储存到数据库中，并提示用户签到成功，可以开始使用该座位。



图 21 用户签退

如果该预约状态为“已签到”则在详细信息界面显示签退按钮，用户点击签退按钮后，用户签签退时间被置为当前时间，并将该时间储存到数据库中，并提示用户签退成功，可以离开该座位。

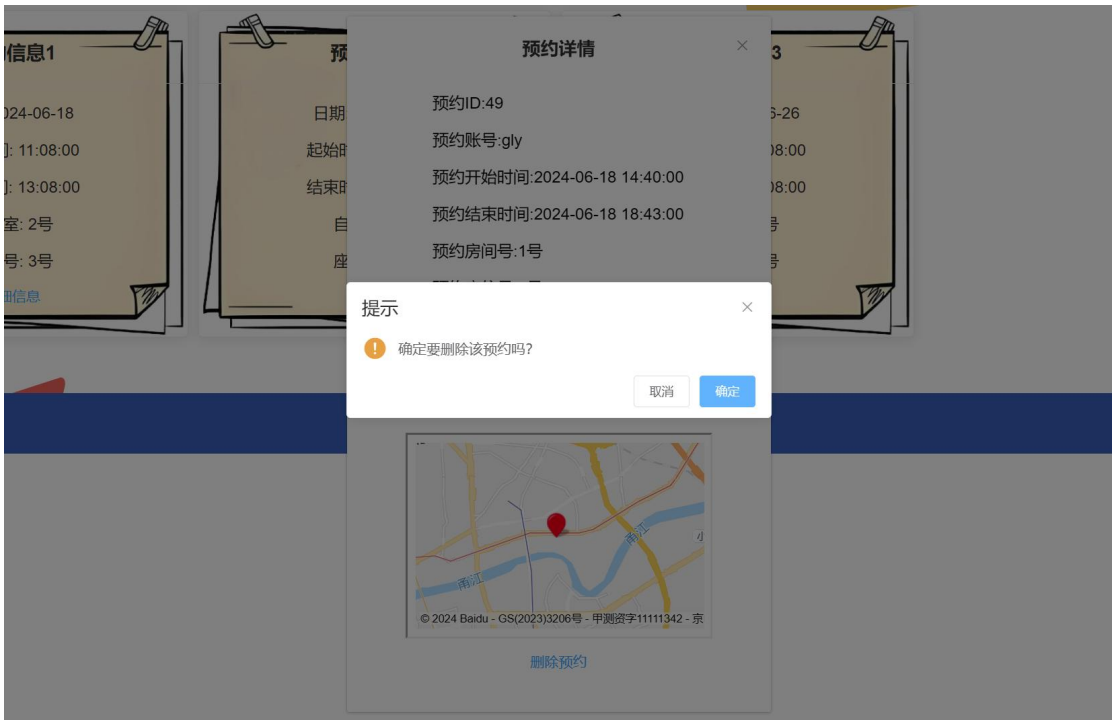


图 22 用户取消预约

在预约状态为“未开始”的时候，用户可以取消该预约，点击删除预约按钮会弹出提示框给予用户二次选择的机会，防止误删除操作，用户点击确认后前端使用 axios 的 delete 函数将该预约信息的 Id 传送到后端进行删除操作。

4.2.4 图书借阅



图 23 图书借阅界面展示

前端通过返回所有图书信息的接口接受所有图书信息，然后表格展示给用户，用户点击借书后借阅当前图书，点击还书后可以归还图书，如果当前图书已经被借阅则借阅按钮无法点击，用户也不能归还不是自己借阅的图书。

4.3 管理员界面



图 24 管理员界面

整个管理员界面的结构由抬头，侧边栏，主体组成。抬头含有智慧图书馆 logo，标题和退出按钮；侧边栏含有不同的管理模块：“自习室管理”，“座位管理”，“预约管理”，“用户管理”，“违规管理”，“图书管理”；主体展示各数据表数据。

4.3.1 自习室管理

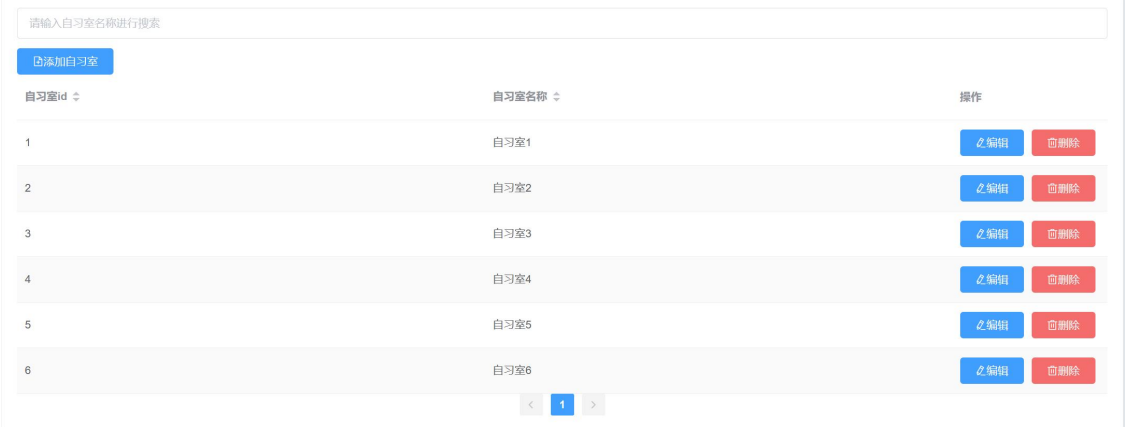


图 25 自习室管理界面

点击“自习室管理”后，前端使用 get 函数调用后端函数拉取数据库自习室表的数据，并显示在前端界面，管理员可以对自习室进行操作，可以修改自习室名称、删除自习室、添加新的自习室或查询自习室，并更新当前界面。

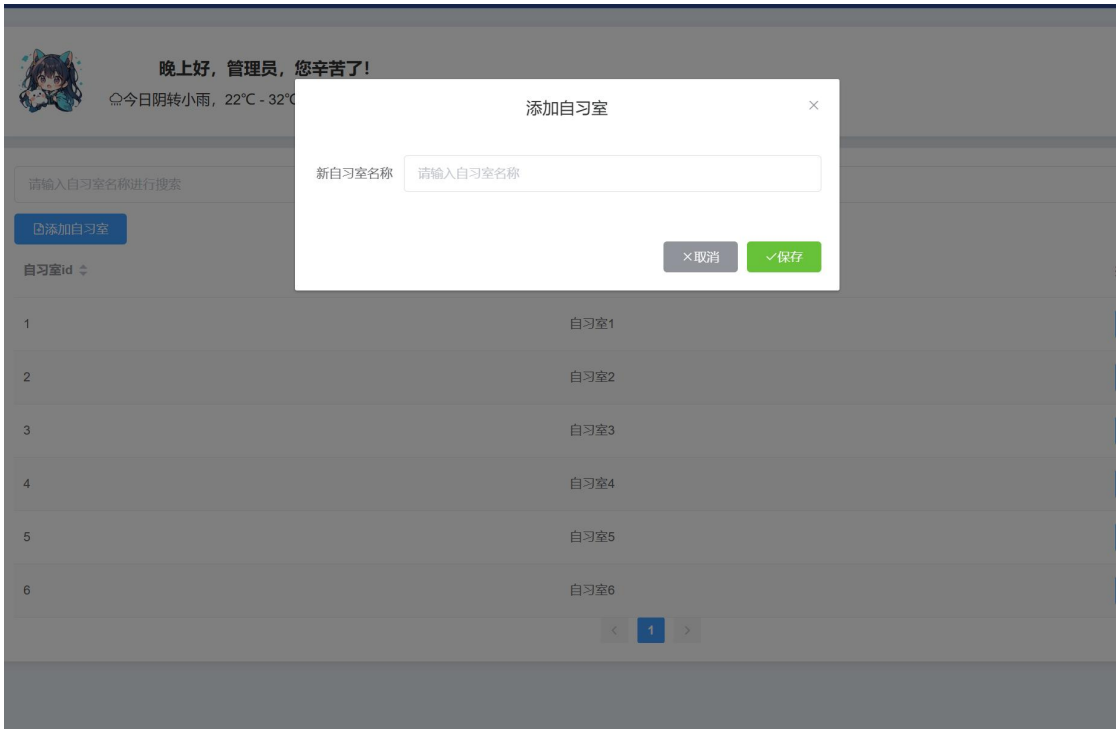


图 26 管理员添加自习室

点击“添加自习室”按钮后，弹出一个弹窗，管理员可以填写“新自习室名称”，然后点击保存，前端就会使用 post 函数调用后端接口，将新自习室传入自习室表中。

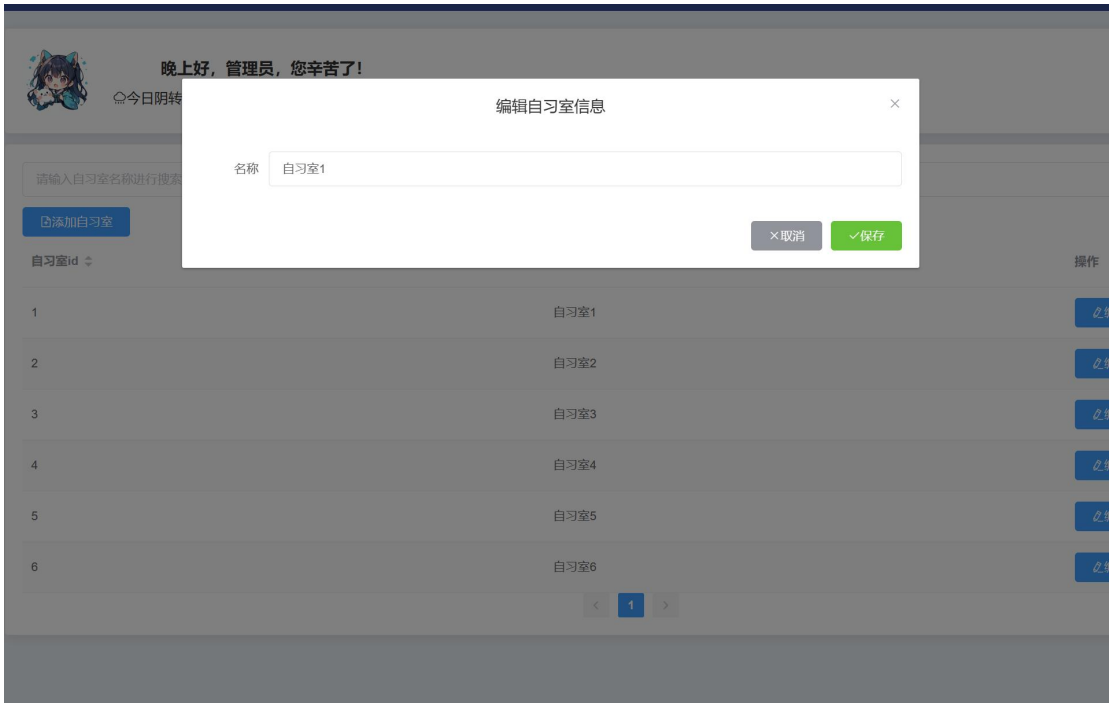


图 27 管理员编辑自习室

点击“编辑”按钮后，弹出一个弹窗，名称栏初始被设定为当前编辑自习室的名字，管理员可以更改“名称”，然后点击保存，如果“名称”不为空，前端就会使用 put 函数调用后端接口，更新对应自习室的名称。

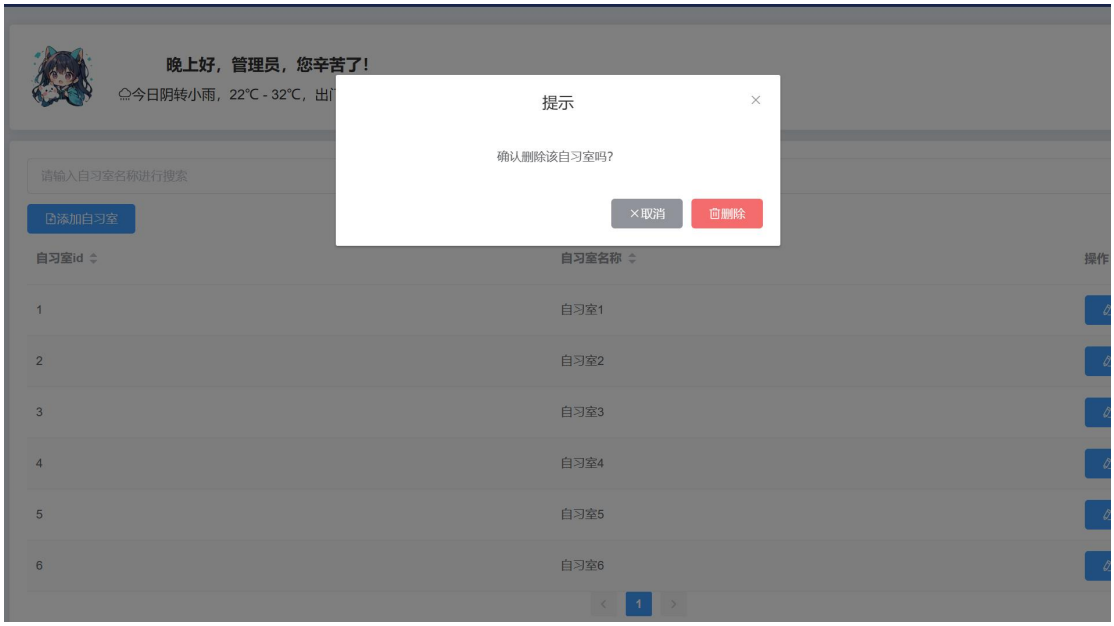


图 28 管理员删除自习室

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该自习室，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用自习室 Id 查找到并删除该自习室。



图 29 管理员查询自习室

管理员在搜索输入框输入自习室 Id 或自习室名称，前端经过过滤器将数据过滤后显示出来，上图可见输入 1 后能够搜索出自习室 Id 或自习室名称含有 1 的自习室。

4.3.2 座位管理



图 30 座位管理界面

点击“座位管理”后，前端使用 get 函数调用后端函数拉取数据库座位表的数据，并显示在前端界面，管理员可以对座位进行操作，可以修改座位、删除座位、添加新的座位或查询座位。

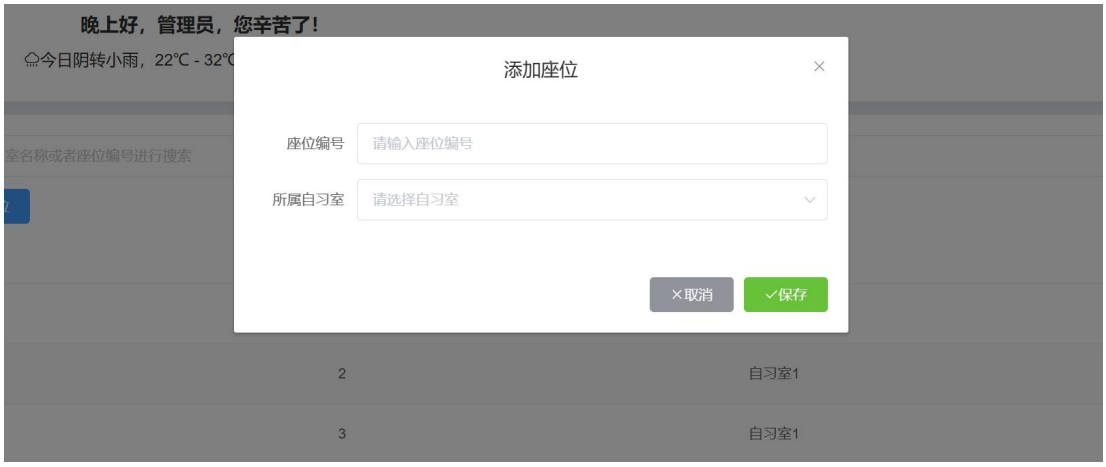


图 31 管理员添加座位

点击“添加座位”按钮后，弹出一个弹窗，管理员可以填写“座位编号”，“所属自习室”，然后点击保存，如果“座位编号”和“所属自习室”不为空，则前端就会使用 post 函数调用后端接口，将新座位传入座位表中，并更新当前界面。

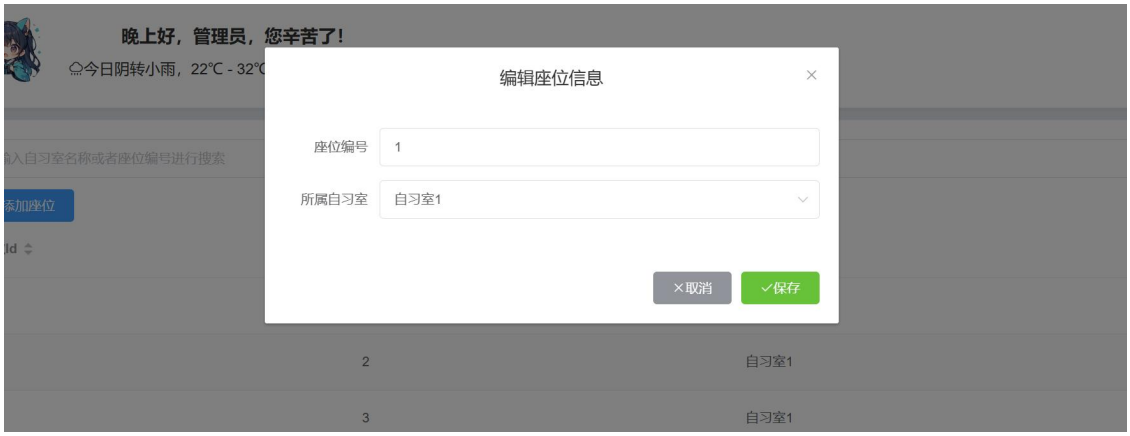


图 32 管理员编辑座位

点击“编辑”按钮后，弹出一个弹窗，座位编号栏初始被设定为当前编辑座位的编号，所属自习室栏被设定为当前编辑座位所属的自习室，管理员可以更改“座位编号”和“所属自习室”，然后点击保存，如果输入栏不为空，前端就会使用 put 函数调用后端接口，更新对应座位的数据。



图 33 管理员删除座位

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该座位，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用座位 Id 查找到并删除该座位。

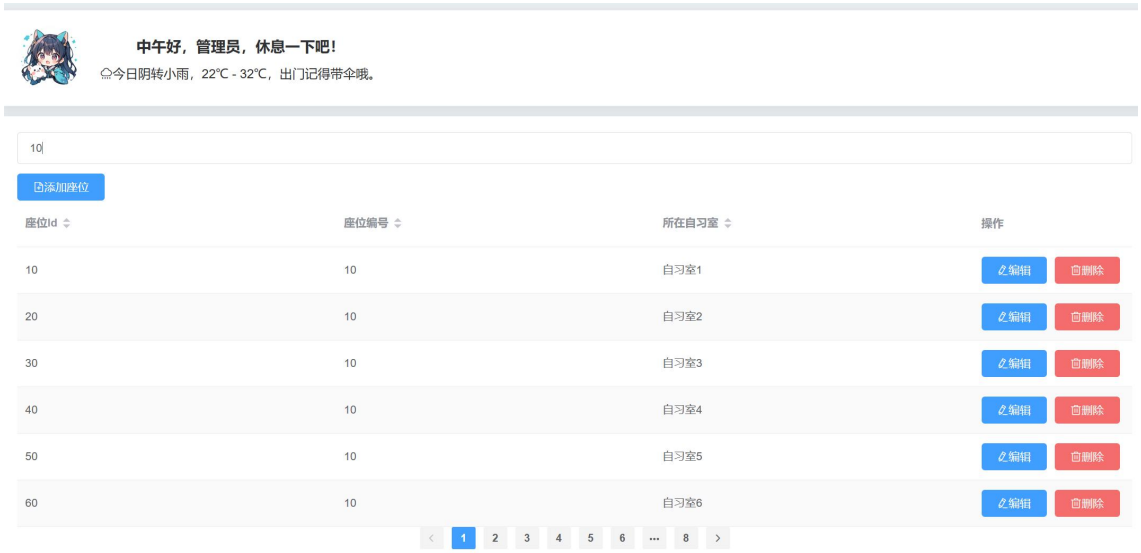


图 34 管理员查询座位

管理员在搜索输入框输入座位编号或所属自习室名称，前端经过过滤器将数据过滤后显示出来，上图可见输入 10 后能够搜索出座位编号为 10 的座位。

4.3.3 预约管理

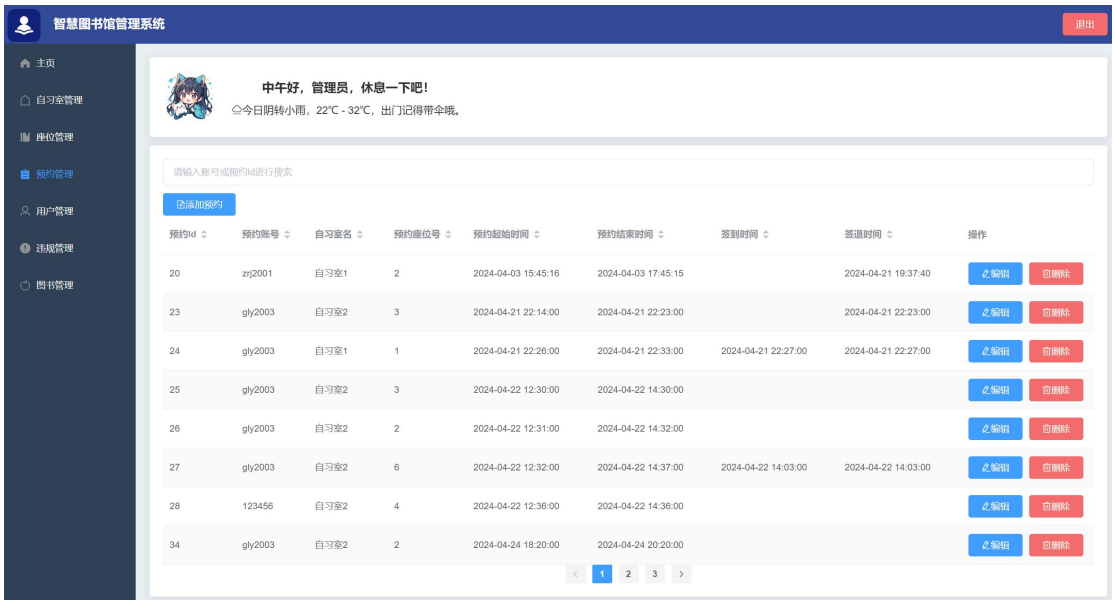


图 35 预约管理界面

点击“预约管理”后，前端使用 get 函数调用后端函数拉取数据库预约信息表的数据，并显示在前端界面，管理员可以对预约信息进行操作，可以修改预约信息、删除预约信息、添加新的预约或查询预约信息。

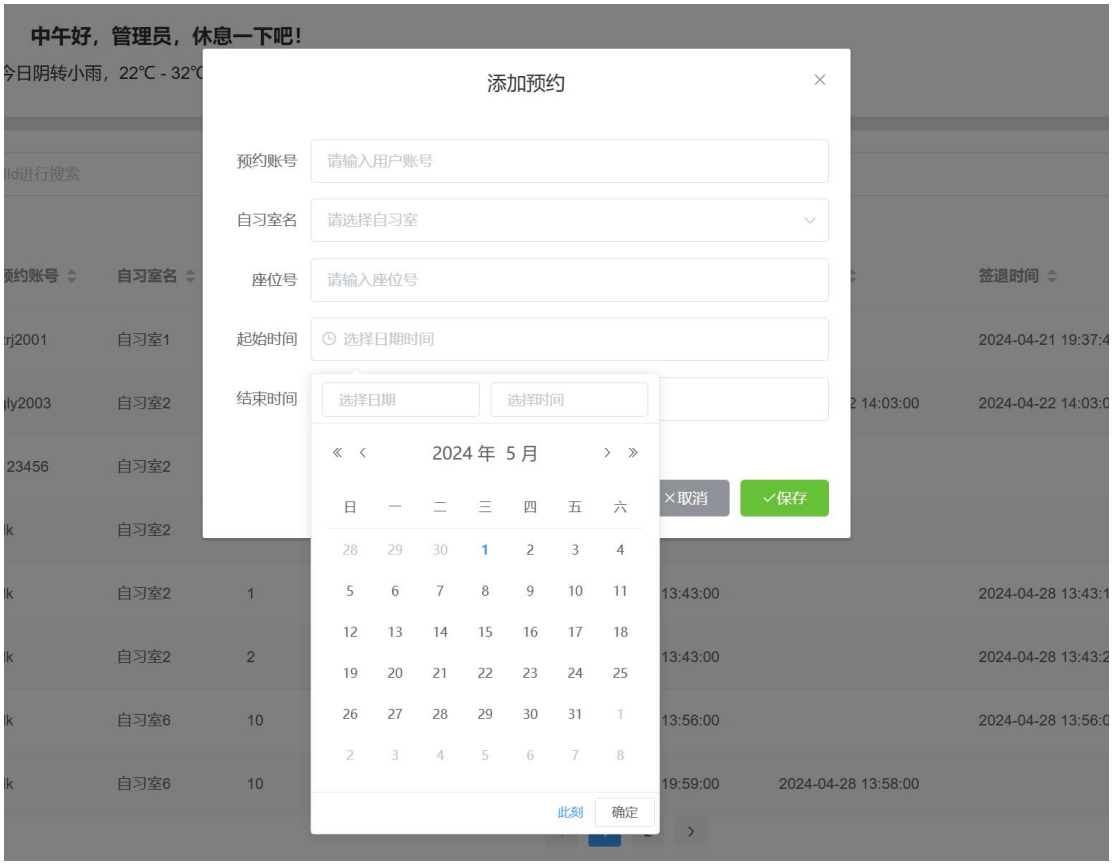


图 36 管理员添加预约信息

点击“添加预约”按钮后，弹出一个弹窗，管理员可以填写“预约账号”，“自习室名”，“座位号”，“起始时间”，“结束时间”，然后点击保存，如果输入内容不为空，则前端就会使用 post 函数调用后端接口，将新预约信息传入预约信息表中，并更新当前界面。



图 37 管理员编辑预约信息

点击“编辑”按钮后，弹出一个弹窗，各输入框内容被默认设置为所编辑的预约信息的内容，管理员可以更改预约信息各项数据，然后点击保存，如果输入栏不为空，前端就会使

用 put 函数调用后端接口，更新对应预约信息的数据。



图 38 管理员删除预约信息

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该预约，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用预约 Id 查找到并删除该预约信息。

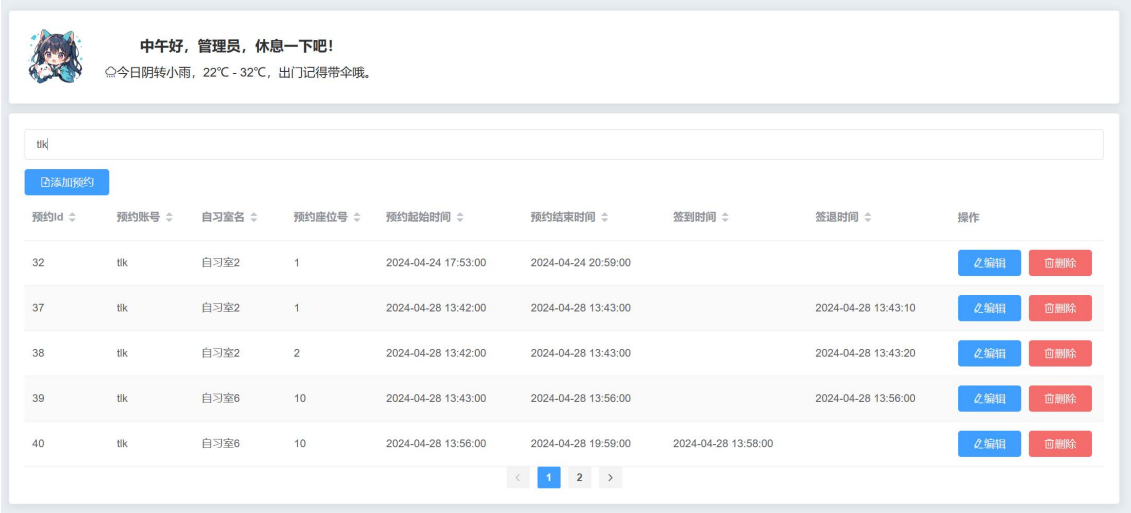


图 39 管理员查询预约信息

管理员在搜索输入框输入预约 Id 或预约账号，前端经过过滤器将数据过滤后显示出来，上图可见输入 tlk 后能够搜索出预约账号为 tlk 的所有预约信息。

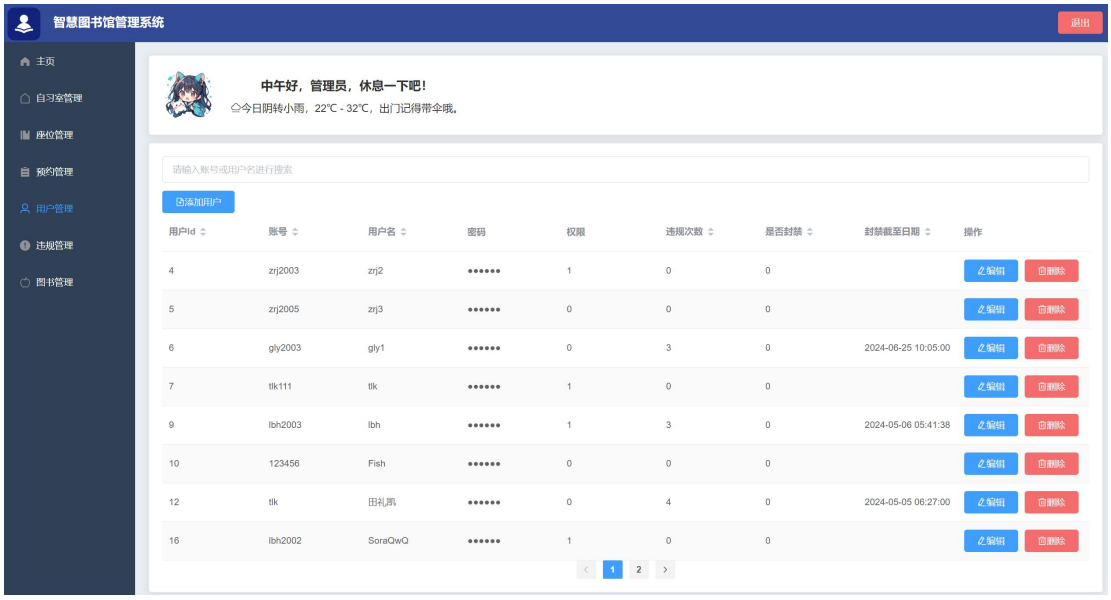


图 40 用户管理界面

点击“用户管理”后，前端使用 get 函数调用后端函数拉取数据库用户表的数据，并显示在前端界面，管理员可以对用户信息进行操作，可以修改用户信息名称，删除用户信息或添加新的用户信息。

4.3.4 用户管理



图 41 管理员添加用户

点击“添加用户”按钮后，弹出一个弹窗，管理员可以填写“用户名”，“账号”，“密码”，“权限”，然后点击保存，如果“用户名”，“账号”，“密码”不为空，则前端就会使用 post 函数调用后端接口，将新用户传入用户表中，并将权限默认设置为 0，代表普通用户。

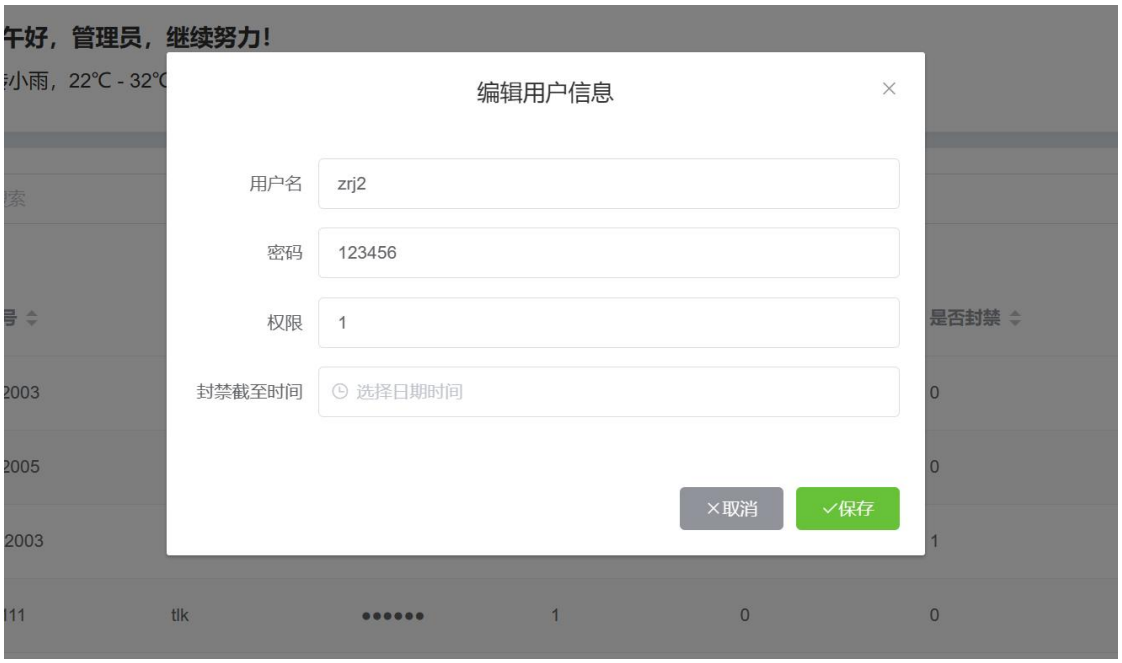


图 42 管理员编辑用户

点击“编辑”按钮后，弹出一个弹窗，各输入框被设定为该用户默认信息，管理员可以更改用户信息，然后点击保存，如果输入栏不为空，前端就会使用 put 函数调用后端接口，更新对应用户的数据。



图 43 管理员删除用户

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该用户，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用账号查找到并删除该用户。

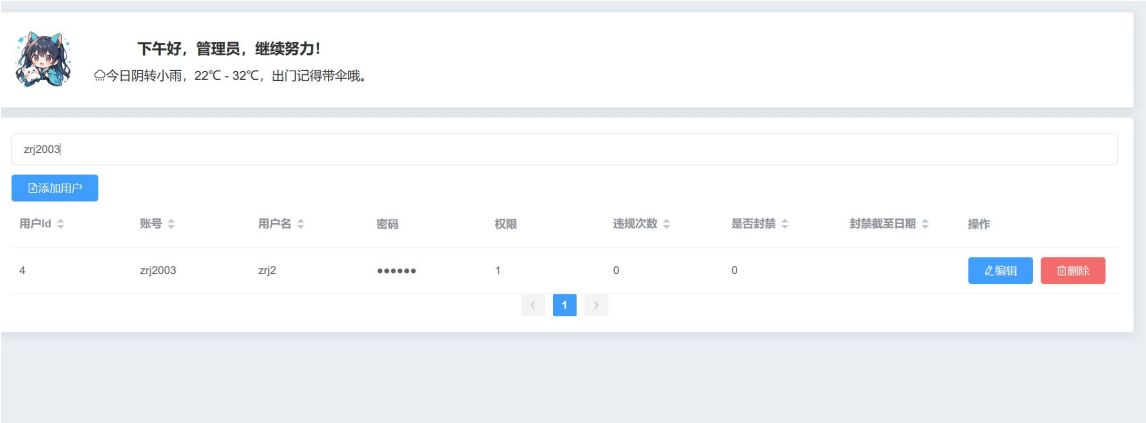


图 44 管理员查找用户

管理员在搜索输入框输入用户名或账号，前端经过过滤器将数据过滤后显示出来，上图可见输入 zrj2003 后能够搜索出账号为 zrj2003 的用户。

4.3.5 违规管理

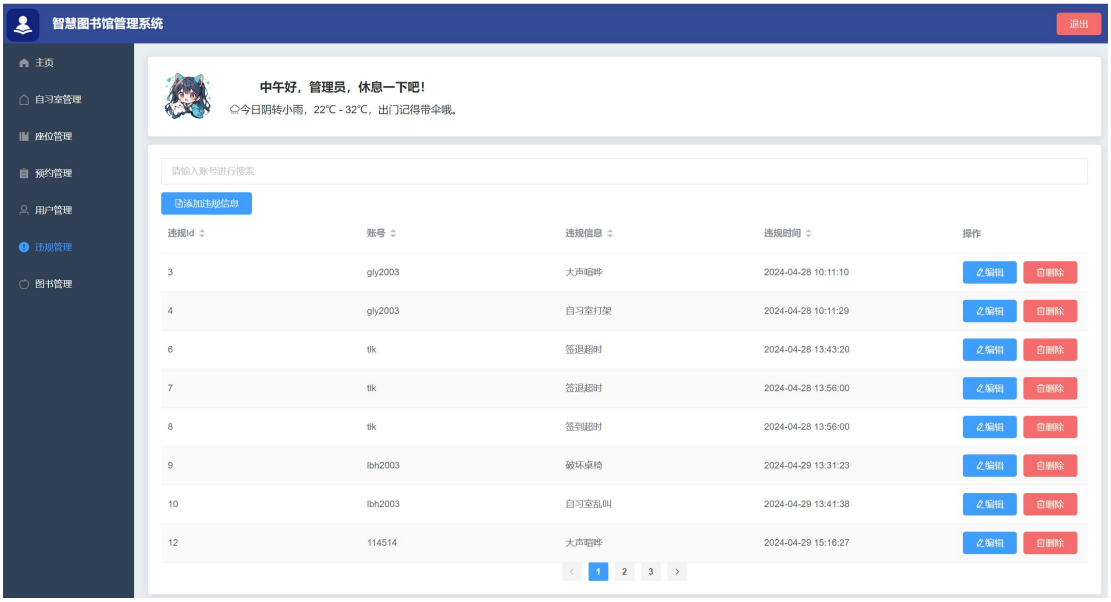


图 45 违规管理界面

点击“违规管理”后，前端使用 get 函数调用后端函数拉取数据库违规表的数据，并显示在前端界面，管理员可以对违规信息进行操作，可以修改违规信息内容，删除违规信息或添加新的违规信息。



图 46 管理员添加违规信息。

点击“添加违规信息”按钮后，弹出一个弹窗，管理员可以填写“账号”，“违规信息”，然后点击保存，如果“账号”和“违规信息”不为空，则前端就会使用 post 函数调用后端接口，将新违规信息传入违规信息表中，并将该用户的“是否违规”标志置为 1，“封禁时间”设置为当前时间的七天后。



图 47 管理员编辑违规信息

点击“编辑”按钮后，弹出一个弹窗，各输入框被默认设置为该违规信息的内容，管理员可以更改“违规账号”，“违规信息”，“违规时间”，然后点击保存，如果输入栏不为空，前端就会使用 put 函数调用后端接口，更新对应违规信息的数据。



图 48 管理员删除违规信息

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该违规信息，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用违规 Id 查找到并删除该违规信息。

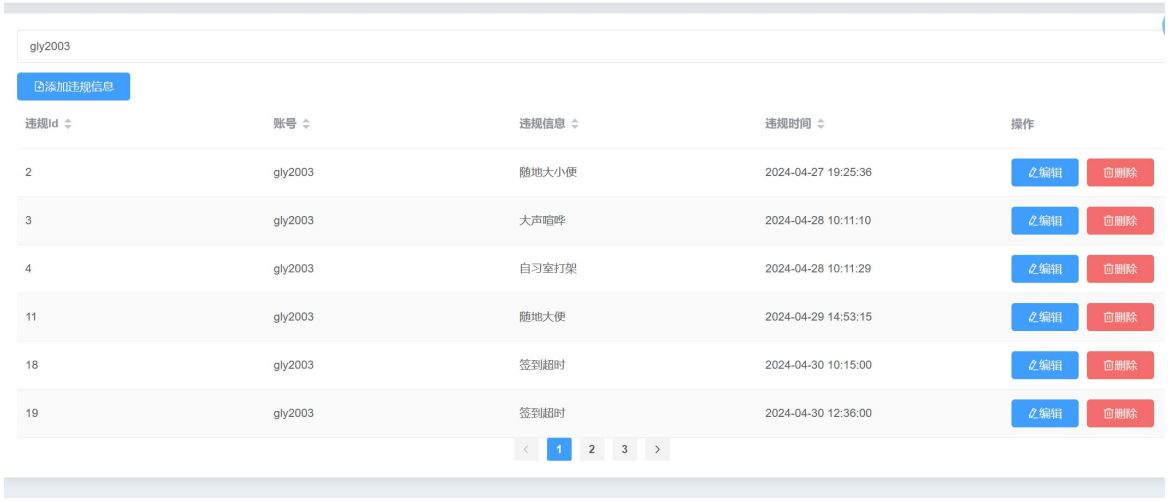


图 49 管理员查询违规信息

管理员在搜索输入框输入违规账号或违规 Id，前端经过过滤器将数据过滤后显示出来，上图可见输入 gly2003 后能够搜索出账号为 gly2003 的所有违规信息。

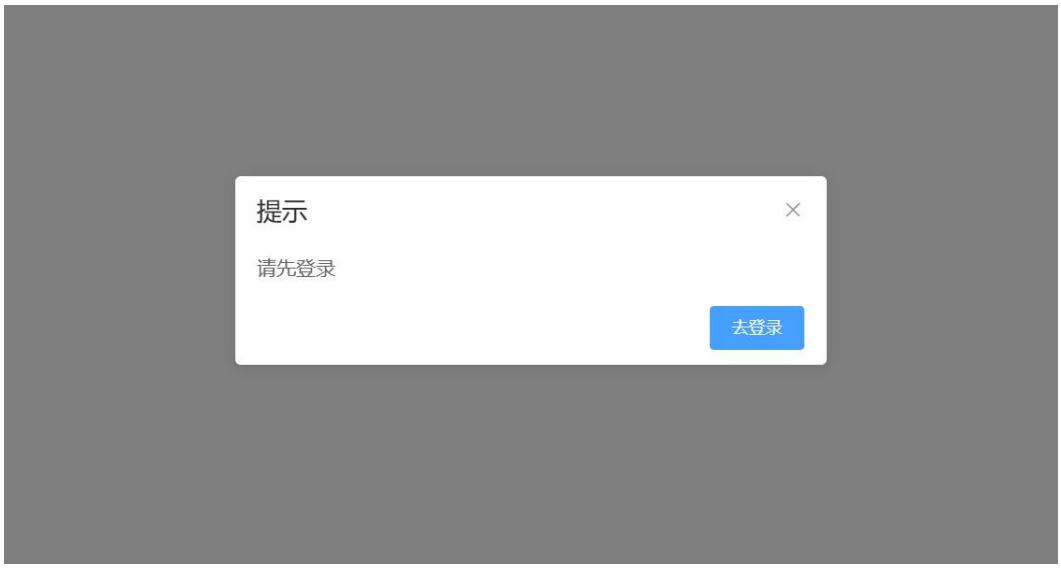


图 50 用户鉴权

如果用户正常登录，则前端将用户账号保存在浏览器会话期间的 sessionStorage 对象，路由器设置中读取该对象取出账号值，判断权限后跳转到预约界面或管理员界面，如果未登录或权限不对则会阻止用户继续访问，从而实现各网页间鉴权。

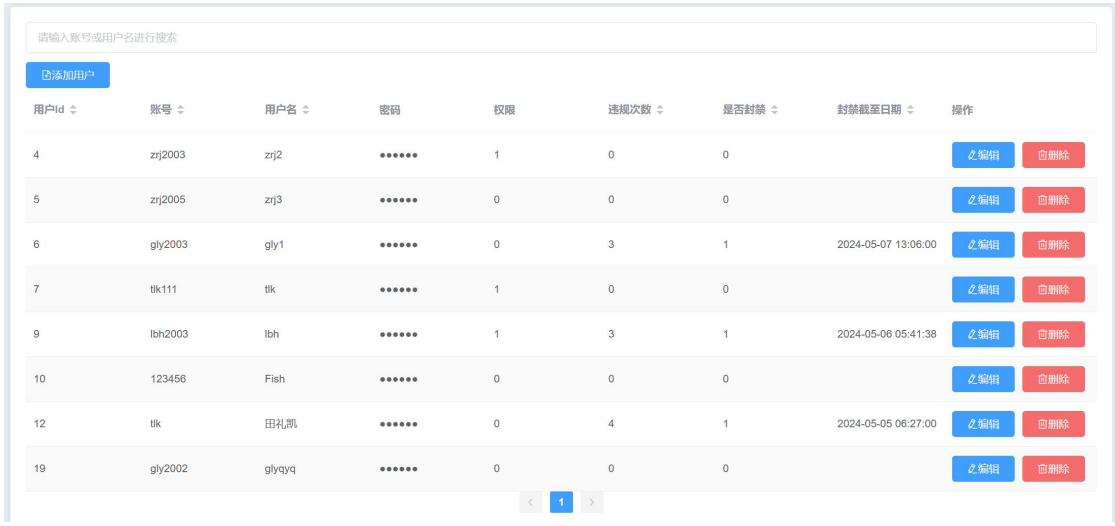


图 51 用户封禁

在用户管理界面我们添加了三列数据，“违规次数”，“是否封禁”，“封禁时间”，如果该用户违规，就将其违规次数加 1，并封禁该用户，封禁时间置为违规时间加 7 天。

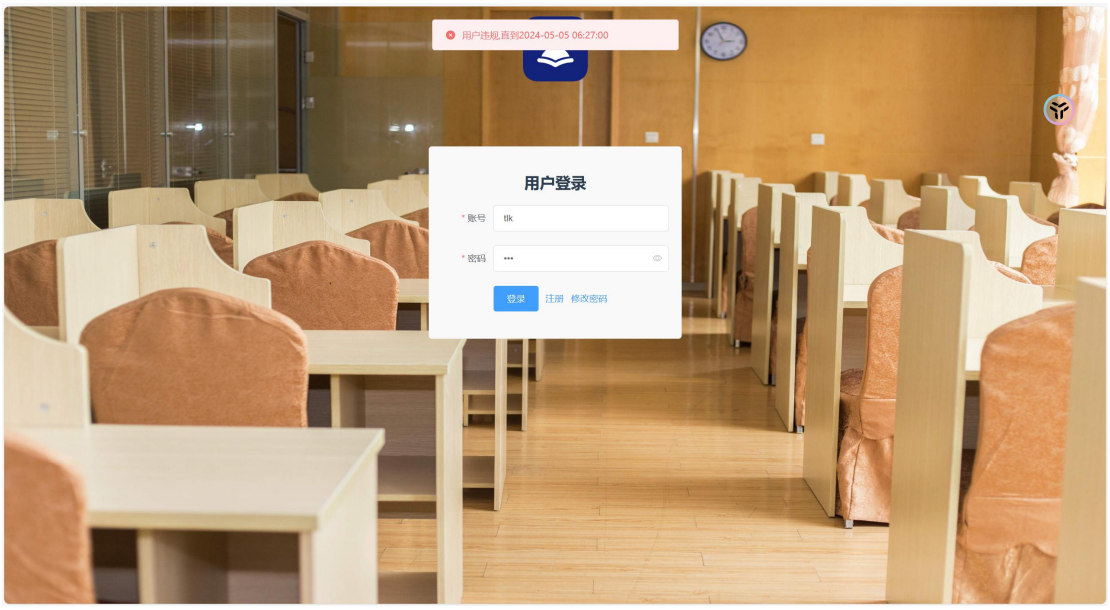


图 52 封禁用户无法登入

前端界面如果检测到该用户“是否封禁”标志为 1，则提示该用户无法登录该智慧自习室，并显示封禁截至时间。

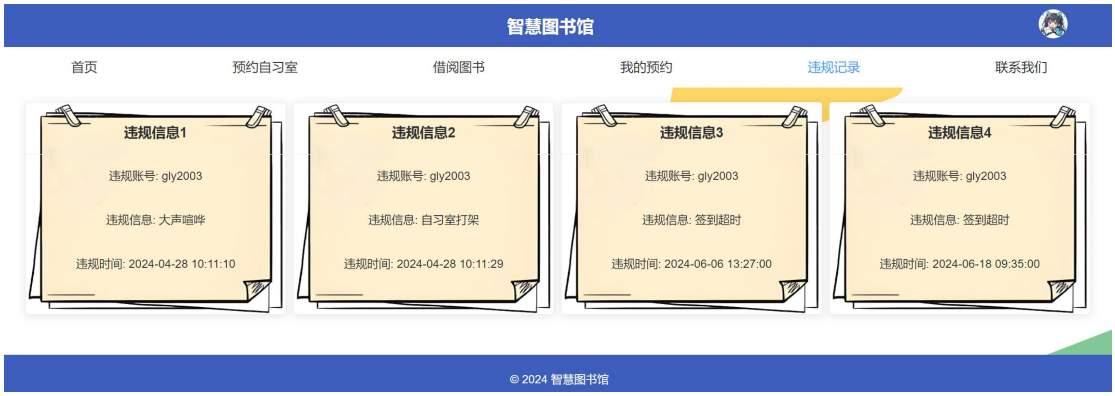


图 53 违规信息展示

点击“违规记录”后，前端使用 get 函数调用后端接口，展示违规信息表数据，用户封禁结束后，可以在“违规记录”下看到自己何时违规的记录。

4.3.6 图书管理



图 54 图书管理界面

点击“图书管理”后，前端使用 get 函数调用后端函数拉取数据库图书表的数据，并显示在前端界面，管理员可以对图书进行操作，可以修改图书信息、删除图书、添加新的图书或查询图书，并更新当前界面。



图 55 管理员添加图书

点击“添加图书”按钮后，弹出一个弹窗，管理员可以填写新图书的相关信息，然后点击保存，前端就会使用 post 函数调用后端接口，将新图书传入图书表中。



图 56 管理员编辑图书

点击“编辑”按钮后，弹出一个弹窗，管理员可以更改图书信息，然后点击保存，前端就会使用 put 函数调用后端接口，更新对应图书。



图 57 管理员删除图书

点击“删除”按钮后，弹出一个弹窗，询问管理员是否确定删除该图书，如果管理员点击确定，前端就会使用 delete 函数调用后端接口，使用图书 Id 查找到并删除该图书。



图 58 管理员查询图书

管理员在搜索输入框输入图书名称，前端经过过滤器将数据过滤后显示出来，上图可见输入“时间”后能够搜索出时间简史这本书。

五、实验总结

项目成功

智慧图书自习室预约管理系统的开发项目在软件实习中取得了显著的成功。项目团队遵循敏捷开发方法和 DevOps 实践，利用华为云 CodeArts 开发平台，实现了从设计到部署的全周期管理。系统的成功不仅体现在功能的完整性上，还表现在它如何显著提升了图书馆服务的智能化水平。通过实现用户登录、注册、座位预约等功能，系统极大地优化了图书馆资源的分配和利用，减少了学生等待时间，提高了管理效率。此外，系统的用户友好界面和流畅的交互体验也得到了用户的广泛认可。

技术实现

技术层面上，项目采用了当前流行的开发技术栈，后端 Springboot 框架和前端 Vue.js 框架的结合，确保了系统的高性能和良好的用户体验。系统架构遵循了清晰的三层架构模式，将 Controller、Service、Mapper 的职责明确分离，提高了代码的可读性和可维护性。数据库操作通过 MyBatisPlus 框架进行简化，减少了开发工作量。前后端分离的开发模式进一步提高了开发效率，使得前端可以独立于后端进行开发和测试。持续集成和持续部署的流水线自动化了代码检查、编译构建和部署流程，这些实践显著提升了开发流程的效率，确保了软件的稳定性和可靠性。

总体评价

综合评价，智慧图书自习室预约管理系统的开发项目不仅是教学与实践相结合的典范，也是校园数字化转型的重要里程碑。项目成功实现了预定目标，提供了一个高效、稳定且用户友好的软件产品，满足了校园内数字化和智能化转型的需求。通过这个项目，学生团队获得了宝贵的实战经验，提升了解决实际问题的能力。同时，系统的实施为图书馆管理带来了创新，提高了服务效率和质量，对学生的学术支持和个人发展产生了积极影响。展望未来，项目团队将继续收集用户反馈，不断迭代和完善系统，探索引入更多创新技术，以满足不断变化的需求，推动智慧图书馆服务向更高水平发展。